UNIVERSITY OF RWANDA

College of Science and Technology



Bachelor of Computer Engineering, Year 4 Semester 1

Real-Time Systems

Unit 3 Laboratory Report

Mini Project 3.1 - Concurrency, Synchronisation and Race Conditions in Python
Mini Project 3.2 - Bounded-Buffer Producer-Consumer Deadlock and Lock-Ordering Fix
Mini Project 3.3 - Priority-Based Interrupt Latency Simulator

Field	Detail
Student names	**Blandon ASHIMIRWE** **Gilbert NTWALI HIRWA** **Janvier NSHIMIYIMANA**
Registration number	**223026675** **223026679** **223026689**
Module	Real-Time Systems
Unit	Unit 3 - RTOS Concurrency, Synchronisation and Interrupt Handling
Submission date	3 November 2026
Prepared with	Python 3.12.10 (standard library only) on Windows 11

GITHUB REPOSITORY LINK: **https://github.com/Blandon2003/RTS_MP_Unit3_Lab.git**

Table of Contents

1. Introduction and Experimental Environment 4
   1.1 Tools, environment and how the timings were made trustworthy 4
   1.2 Validity of the Python sandbox 4
2. Mini Project 3.1 - Concurrency, Synchronisation and Race Conditions 5
   2.1 Objective 5
   2.2 Scripts produced 5
   2.3 The race condition and its repair 5
   2.4 Bounded queue as an RTOS message queue 6
   2.5 Counting semaphore as a fixed-size resource pool 7
   2.6 Dispatch latency and synchronisation overhead 7
   2.7 Answers to the Lab Tasks 8
      2.7.1 Lab Task 1 - ten threads instead of five 8
      2.7.2 Lab Task 2 - semaphore count 1 and count 6 8
      2.7.3 Lab Task 3 - livelock with two threads acquiring two locks in opposite order 9
      2.7.4 Lab Task 4 - queue latency against maxsize 9
      2.7.5 Lab Task 5 - why threading.Semaphore cannot replace threading.Lock 9
3. Mini Project 3.2 - Bounded-Buffer Deadlock and the Lock-Ordering Fix 11
   3.1 Objective and system model 11
   3.2 Coffman's four conditions mapped to the flawed code 11
   3.3 Deadlock reproduction 11
   3.4 The lock-ordering fix and its verification 12
   3.5 Throughput against buffer capacity 12
   3.6 Analysis 13
4. Mini Project 3.3 - Priority-Based Interrupt Latency Simulator 15
   4.1 Objective and design 15
   4.2 Results per configuration 15
      4.2.1 Baseline - short top halves throughout 15
      4.2.2 Long top half on IRQ_HIGH (12 ms) 15
      4.2.3 Burst arrivals - IRQ_HIGH period 15 ms 16
   4.3 Configuration comparison 16
   4.4 Preemption evidence 16
   4.5 Arrival-rate sweep 17
   4.6 Analysis 18
5. Conclusions 20
   5.1 Deliverable checklist 20
6. References 20

Appendix A - Mini Project 3.1 source 22
A.1 race_demo.py - unsaturated shared counter 22
A.2 race_fixed.py - mutex-protected counter 22
A.3 queue_sim.py - bounded message queue 23
A.4 semaphore_pool.py - counting-semaphore resource pool 25
A.5 timing_overhead.py - dispatch latency and lock overhead 27
A.6 livelock_demo.py - Lab Task 3 29
Appendix B - Mini Project 3.2 source 31
B.1 buffer_flawed.py - inverted lock order 31
B.2 buffer_fixed.py - global lock ordering 34
B.3 capacity_sweep.py - throughput versus capacity 37
Appendix C - Mini Project 3.3 source 40
C.1 interrupt_latency_sim.py - priority interrupt simulator 40

If the entries above are not populated, select them and press F9 (or right-click > Update Field) in Word.

1. Introduction and Experimental Environment

This report documents the three Unit 3 mini projects of the Real-Time Systems laboratory. Each project uses Python's threading module as a controlled sandbox for a real-time operating-system concept: Mini Project 3.1 reproduces and repairs the race condition that arises when tasks share state without mutual exclusion, models an RTOS message queue with a bounded queue.Queue, and measures dispatch latency and synchronisation overhead; Mini Project 3.2 builds a multi-producer, multi-consumer bounded buffer that deliberately satisfies Coffman's four necessary conditions for deadlock, detects the deadlock with a watchdog thread, repairs it with a global lock-ordering discipline, and then measures throughput against buffer capacity; and Mini Project 3.3 simulates three periodic interrupt sources with top-half and bottom-half work scheduled by strict priority, measuring interrupt latency and response latency for well-behaved and badly-behaved interrupt service routines.

The three projects are deliberately cumulative. Mini Project 3.1 establishes the vocabulary of the unit - tasks, blocking, mutual exclusion, signalling and queues - and shows that a concurrency bug is reproducible and measurable. Mini Project 3.2 shows that correct use of mutexes is not sufficient, because the order in which they are acquired is itself a design decision. Mini Project 3.3 moves from correctness to timing, showing that even a deadlock-free, correctly synchronised system can miss its deadlines if priority levels are used badly.

1.1 Tools, environment and how the timings were made trustworthy

- Python 3.12.10, standard library only (threading, time, queue, random, statistics, heapq, ctypes, argparse, csv, json). No third-party runtime dependency is used by any simulation; matplotlib 3.10 is used only to draw the charts from the captured data.
- Windows 11 (10.0.22631), single machine, no other load applied while measurements were taken.
- Windows exposes sleep and thread-wake granularity of about 15.6 ms by default, which is coarser than every timing quantity this lab measures. Each script therefore calls winmm.timeBeginPeriod(1) through ctypes for the duration of the run and restores it with timeEndPeriod(1); all millisecond figures below depend on that single call.
- All timings are wall-clock differences from time.perf_counter(), reported in milliseconds to the precision the platform can actually support.
- Every console figure quoted in this report was produced by the batch runner run_evidence.sh, which executes the scripts in sequence and writes each transcript to the logs folder; the results folder holds the matching CSV and JSON artefacts.

1.2 Validity of the Python sandbox

Two properties of CPython limit how literally these experiments may be read as RTOS behaviour, and both are stated here rather than buried in the conclusions. First, the Global Interpreter Lock prevents two threads from executing bytecode simultaneously, so the concurrent threads model concurrent RTOS tasks that interleave on one CPU, not tasks running on separate cores. Second, CPython has no task priorities: the operating system scheduler chooses which runnable thread executes. Mini Project 3.1's experiments are unaffected because they concern correctness under interleaving, and Mini Project 3.3 recovers priority behaviour explicitly by making every bottom half yield to higher-priority ready work and by treating the CPU as a mutex, so strict priority is enforced by the simulator's own scheduling loop instead of by the host scheduler. Where the host scheduler still leaks into the results - the dispatch-latency tail in section 2.6, for instance - it is identified as such.

2. Mini Project 3.1 - Concurrency, Synchronisation and Race Conditions

2.1 Objective

To reproduce, diagnose and repair the concurrency hazards of Unit 3 in a controlled way: multiple tasks contending for shared state (Section 3.8); mutual exclusion through a mutex-like lock and signalling through a counting semaphore (Section 3.4); a bounded shared buffer standing in for an RTOS message queue (Section 3.4); and timing measurements that approximate context-switch and interrupt-response cost (Sections 3.2 and 3.5). A livelock demonstration was added for Lab Task 3.

2.2 Scripts produced

Script	Models	What it demonstrates
race_demo.py	Five or ten tasks sharing a counter	Lost-update race; non-repeatable result
race_fixed.py	Same, with threading.Lock	Mutual exclusion restores the expected count
queue_sim.py	queue.Queue(maxsize=N) message queue	Producer blocking and per-item latency
semaphore_pool.py	threading.Semaphore(3) resource pool	Counting-semaphore admission and wait time
timing_overhead.py	Thread dispatch plus shared mutex	Worst-case dispatch latency; overhead scaling
livelock_demo.py	Two tasks, opposite lock orders, retrying	Livelock: activity without progress

Table 2.1 - The five required scripts plus the Lab Task 3 script.

2.3 The race condition and its repair

race_demo.py defines shared_counter at module scope with no lock. Each task runs increment_unsafe(), which reads the counter, sleeps 0.1 ms to widen the interleaving window, and writes back the incremented value - the three steps of a read-modify-write that a real RTOS would protect either by disabling interrupts or by taking a mutex. race_fixed.py is the same program with counter_lock = threading.Lock() wrapping those three steps in a with counter_lock: block.

Three consecutive runs of each configuration are reproduced below. Five tasks each perform 1000 increments, so the expected final value is 5000; the ten-task Lab Task run expects 10 000.

Run	Threads	Expected	Actual	Lost updates	Error	Elapsed (s)
unsafe 1	5	5000	1926	3074	61.5%	3.91
unsafe 2	5	5000	1943	3057	61.1%	6.99
unsafe 3	5	5000	1810	3190	63.8%	2.52
mutex 1	5	5000	5000	0	0	8.06
mutex 2	5	5000	5000	0	0	5.49
mutex 3	5	5000	5000	0	0	5.47
unsafe 1	10	10000	1000	9000	90.0%	3.34
unsafe 2	10	10000	2865	7135	71.3%	4.45
unsafe 3	10	10000	1002	8998	90.0%	6.17
mutex 1	10	10000	10000	0	0	16.10
mutex 2	10	10000	10000	0	0	17.60
mutex 3	10	10000	10000	0	0	14.38

Table 2.2 - Final shared_counter: three unsynchronised runs at five threads, three mutex-protected runs at five threads, then the same six runs with ten threads (Lab Task 1).

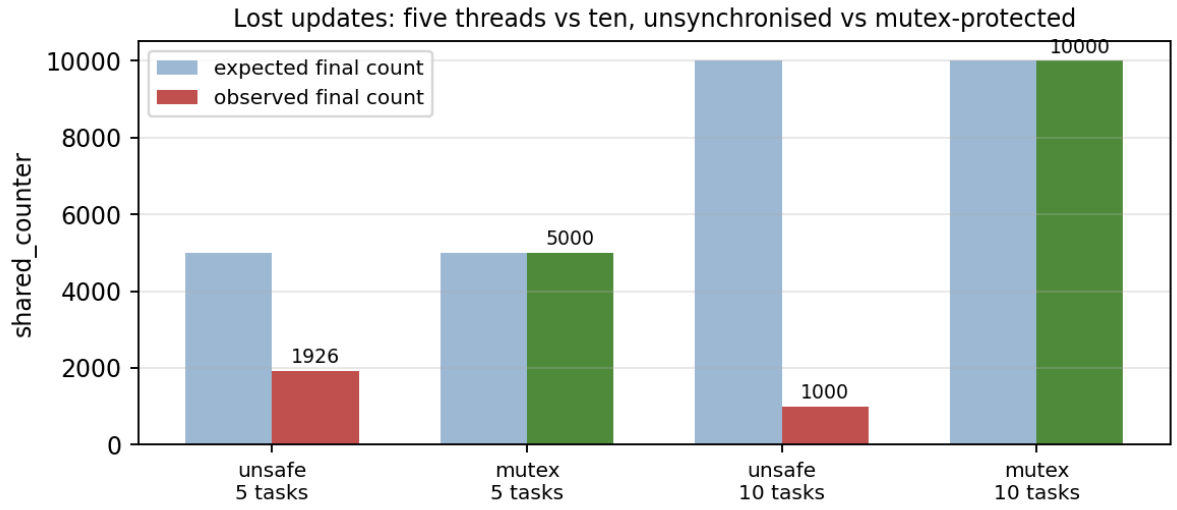


Figure 2.1 - Observed versus expected counter values. The unsynchronised bars lose 61-64 % of the increments at five tasks and 71-90 % at ten The mutex-protected bars land exactly on the expected value in every run.

Console extract - race_demo.py, five threads, run 1 (unsafe) and race_fixed.py, five threads, run 1 (fixed)

```
--- race_demo.py, run 1 (five threads) ---
threads=5  iterations/thread=1000
expected=5000  actual=1926  lost_updates=3074 (61.48%)
elapsed=3.909s

--- race_fixed.py, run 1 (five threads) ---
threads=5  iterations/thread=1000
expected=5000  actual=5000  lost_updates=0
elapsed=8.062s
```

2.4 Bounded queue as an RTOS message queue

queue_sim.py uses queue.Queue(maxsize=5) so that a put() blocks once five items are unread and a get() blocks when the queue is empty, exactly the behaviour of an RTOS message queue with a fixed depth. The producer timestamps each item before the put(); the consumer timestamps it after the get() and reports the difference as delivery latency. Two runs of thirty items arriving every 10 ms are compared: a consumer that does no work per item, and a consumer that spends 4 ms per item.

Consumer service	Min (ms)	Mean (ms)	p95 (ms)	Max (ms)	Blocking puts	Producer blocked (ms)
0.0 ms	0.081	0.152	0.354	0.379	0	0.000
12.0 ms	0.358	25.320	59.840	61.458	0	0.000

Table 2.3 - Send-to-receipt latency and producer blocking for a five-deep queue, thirty items, fast and slow consumers.

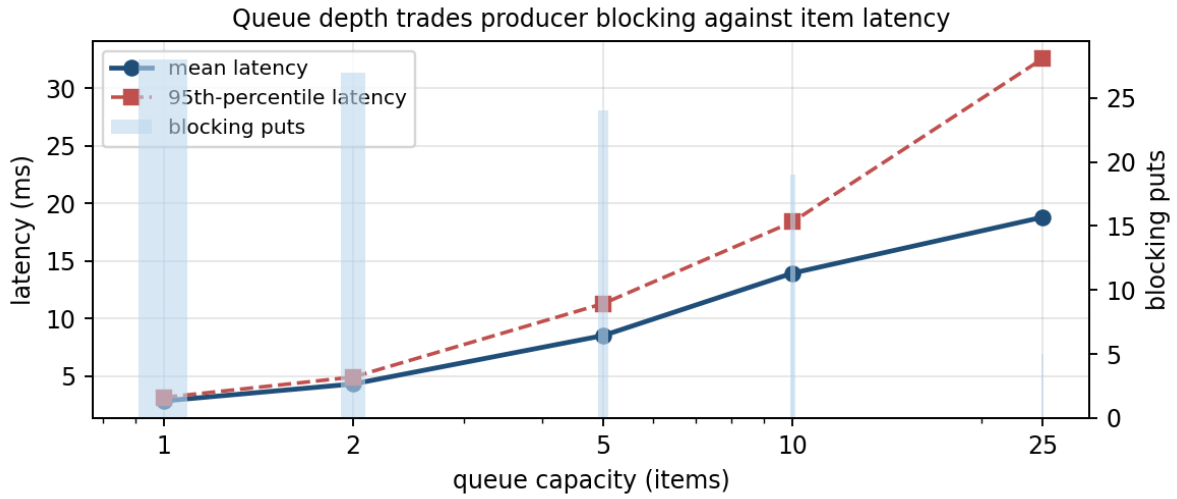


Figure 2.2 - Capacity sweep (Lab Task 4): mean and 95th-percentile item latency against queue depth, with the number of blocking producer puts shown as bars.

2.5 Counting semaphore as a fixed-size resource pool

semaphore_pool.py creates pool_semaphore = threading.Semaphore(3), the analogue of three identical fixed-size resource blocks such as DMA buffers or communication channels. Eight tasks are launched; each records the time it spent blocked in acquire(), holds the resource for a random interval between 20 and 60 ms, then calls release(). A random seed is fixed so the hold durations repeat exactly.

Task	Wait before acquire (ms)	Hold (ms)	Admitted immediately?
task 0	0.018	23.6	yes
task 1	0.007	26.5	yes
task 2	0.002	27.8	yes
task 3	24.383	59.6	no
task 4	26.563	52.6	no
task 5	28.011	22.2	no
task 6	50.378	30.2	no
task 7	80.856	49.6	no

Table 2.4 - Semaphore count 3, eight tasks: the first three tasks proceed without blocking, the remaining five accumulate measurable wait time, and no more than three tasks ever hold the resource.

2.6 Dispatch latency and synchronisation overhead

timing_overhead.py measures two things. Part 1 loops 200 times, recording the instant before a thread is started and the instant its threading.Event is observed by event.wait() in the main thread; that interval is the Python analogue of the time from an interrupt being asserted to the first instruction of the handler that services it. Part 2 runs N tasks that each update shared state 300 times while holding the mutex for 0.25 ms per update, and compares the wall clock against the same loop with no lock.

Statistic	Value (ms)
Minimum	0.2585
Mean	0.4579
Median	0.3169
95th percentile	0.8473
Maximum (worst case observed)	6.6337
Standard deviation	0.6167

Statistic	Value (ms)
Worst / mean ratio	14.5x

Table 2.5 - Thread-dispatch latency over 200 events, the same quantity an RTOS report would call interrupt latency.

Tasks	No lock (ms)	Mutex (ms)	Serialisation	Unlocked correct	Unlocked final / expected
1	216.5	288.5	1.33x	True	300 / 300
2	233.5	526.8	2.26x	False	300 / 600
5	222.3	1122.2	5.05x	False	300 / 1500
10	228.2	2356.2	10.33x	False	301 / 3000

Table 2.6 - Cost of holding a mutex versus concurrency, with the unsynchronised run's correctness in the last two columns.

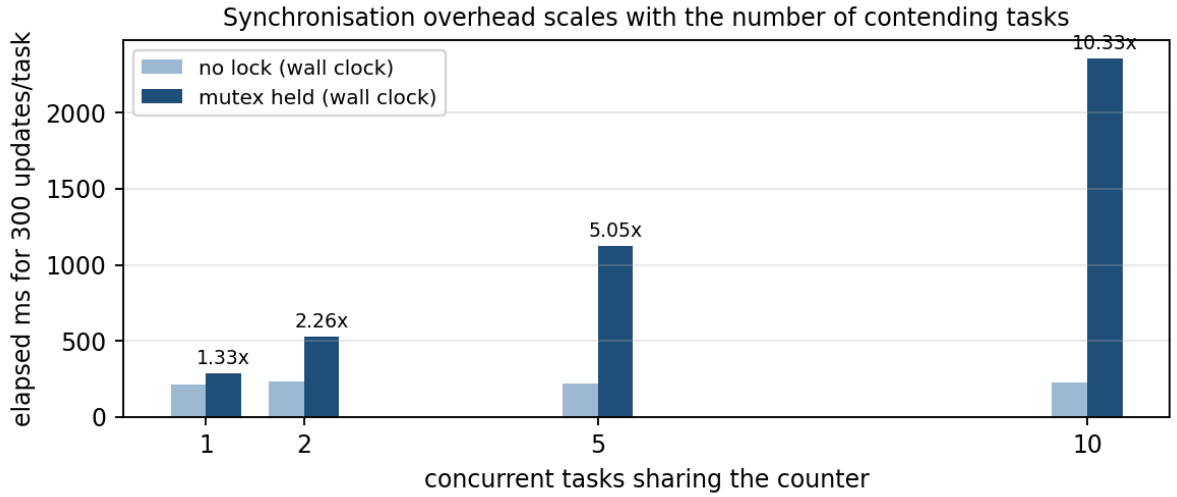


Figure 2.3 - Mutex-held wall clock rises roughly with the number of contending tasks while the unsynchronised wall clock stays flat, because it is doing lost updates rather than useful work.

2.7 Answers to the Lab Tasks

2.7.1 Lab Task 1 - ten threads instead of five

The lost-update error grows. At five threads the three runs of race_demo.py ended at 1926, 1943, 1810 against an expected 5 000, errors of 61.1-63.8 %; at ten threads the runs ended at 1000, 2865, 1002 against an expected 10 000, errors of 71.3-90.0 %. The mechanism is arithmetic rather than mysterious: every increment is a read-modify-write that opens a 0.1 ms window, and with n tasks each window is exposed to n-1 other tasks that may read the same stale value before the write lands. Doubling the contenders roughly doubles the number of stale reads stacked on a single value, and the final counter collapses towards the number of tasks that happened to be the last writer of each distinct value rather than towards the intended total. The result is also less repeatable at ten threads, which is the point about races being non-deterministic. The fixed version reaches 10000 - the expected value exactly - in all three ten-thread runs, at a wall-clock cost.

2.7.2 Lab Task 2 - semaphore count 1 and count 6

Changing the pool size from three to one and to six, with eight tasks and the same seeded hold durations, produces:

count = 1: 1 task admitted immediately, 7 blocked, mean wait 118.615 ms, maximum 247.602 ms, makespan 300.8 ms.

count = 3: 3 admitted immediately, 5 blocked, mean wait 26.205 ms, maximum 79.732 ms, makespan 139.8 ms.

count = 6: 6 admitted immediately, 2 blocked, mean wait 5.767 ms, maximum 23.572 ms, makespan 77.0 ms.

Contention rises steeply as the count falls and vanishes as it approaches the number of tasks. At count 1 the semaphore behaves as a mutex: the maximum simultaneous holders is one, the tasks are fully serialised, the mean wait is roughly half the makespan, and the makespan equals the sum of all hold times. At count 6 only two tasks ever wait, because the resource is larger than the realistic concurrency the eight tasks can present. Section 3.4's counting semaphore is exactly this admission control: the count is the capacity, waiting is proportional to the shortfall between demand and capacity, and raising the count trades memory or hardware cost for waiting time. The interesting detail is that the makespan does not fall proportionally with the count - at count 1 it is the sum of every hold, while at count 6 it approaches the longest single hold chain - which is Amdahl-shaped: the last task to be served still waits behind whoever started last.

2.7.3 Lab Task 3 - livelock with two threads acquiring two locks in opposite order

livelock_demo.py starts two tasks on a barrier, gives them two locks in opposite orders, and has both use non-blocking acquire() calls with an identical 0.5 ms back-off and retry, with a third monitor thread sampling progress. The distinction from Mini Project 3.2's deadlock is that both tasks here remain runnable: neither is blocked, both keep releasing and re-requesting, and the retry counter is the only evidence that anything is wrong.

Measured over a 3 s window: 4621 retries (task0=2312, task1=2309), 0 completed critical sections. Because the two tasks take the opposite order and back off for exactly the same interval, they wake together, collide together and back off together, forever; the symmetric retry is the bug. Running the identical loop with a fixed acquisition order gives 4030 completions and 0 retries in the same window, which isolates the cause as the ordering rather than the retry logic itself.

This is Section 3.8's definition of livelock precisely: the tasks are not blocked and are not idle, the system is expending CPU, yet no forward progress is made. Deadlock shows as zero CPU usage with everything waiting; livelock shows as 100 % CPU usage with everything retrying, which is why it is harder to spot from a utilisation graph and why the retry counter, not the task states, is the diagnostic.

2.7.4 Lab Task 4 - queue latency against maxsize

Thirty items with a 1 ms consumer service time were run at five capacities. The producer submits as fast as it can, so the queue absorbs the difference between the arrival rate and the drain rate:

capacity 1: mean 2.821 ms; capacity 2: mean 4.295 ms; capacity 5: mean 8.504 ms; capacity 10: mean 13.923 ms; capacity 25: mean 18.783 ms (see Table 2.3 and Figure 2.2 for the full min/p95/max and blocking-put columns).

The direction is unambiguous and it is the opposite of the naive intuition that a bigger queue is always better. A shallow queue throttles the producer by blocking puts, which caps the number of items in flight and therefore caps the latency any individual item can suffer; a deep queue lets the producer run ahead, so items pile up behind the consumer and each item's wait grows roughly in proportion to the queue's depth. Blocking puts fall as capacity rises while mean latency climbs. The RTOS reading is that queue depth is a design input, not a safety margin: it sets an upper bound on the buffering component of response time, and a queue that is deep enough never to block is also deep enough to hide an overload condition until it is too late to do anything about it.

2.7.5 Lab Task 5 - why threading.Semaphore cannot replace threading.Lock

A plain counting semaphore would be unsuitable wherever the reasoning depends on who owns a resource, which is exactly the situation Section 3.4 describes for mutexes. A threading.Semaphore(1) enforces the same one-at-a-time admission as a mutex, but it has no notion of ownership: its state is a count, not a holder, so any thread may call release() whether or not it ever called acquire(), and a task that releases a semaphore it does not hold silently raises the count above its initial value and permits two tasks into the critical section at once. A mutex, by contrast, records its owning task, refuses to be released by a non-owner, and in real RTOS kernels supports recursive acquisition and deletion semantics. Ownership is not cosmetic - it is the hook that

priority-inheritance protocols need. If a high-priority task blocks on a resource held by a low-priority task, the kernel boosts the holder's priority, and to do that it must be able to answer the question who is holding this object, which a semaphore count cannot answer. The same ownership record is what makes deadlock detection and lock-use auditing possible at all. For signalling, where the correct primitive is deliberately owner-free - an ISR posting to a consumer, a pool of interchangeable buffers - the counting semaphore is the right tool, and semaphore_pool.py depends on that property; for mutual exclusion it is the wrong one.

3. Mini Project 3.2 - Bounded-Buffer Deadlock and the Lock-Ordering Fix

3.1 Objective and system model

Three producer tasks and three consumer tasks share a bounded buffer and a statistics dictionary that both roles update, each protected by its own explicit lock - buffer_lock for the buffer list, stats_lock for the counters - so that acquisition order is fully under the programmer's control rather than hidden inside queue.Queue. buffer_flawed.py has producers take buffer_lock first and then stats_lock, and consumers take stats_lock first and then buffer_lock. A watchdog thread samples the produced and consumed counters and, after a configurable stall window with no change, prints the progress figures, the per-task lock-wait snapshot and the Coffman conditions in force, then ends the run without a manual kill.

3.2 Coffman's four conditions mapped to the flawed code

Condition	Code that satisfies it
1. Mutual exclusion	buffer_lock and stats_lock are threading.Lock objects, which admit one holder and have no shared mode; the bounded buffer list itself may be touched by only one task at a time.
2. Hold and wait	producer_body() enters buffer_lock and, while still holding it, requests stats_lock to increment the counters; consumer_body() enters stats_lock and, while holding it, requests buffer_lock to pop an item. Both roles then spin for space or for an item without releasing the lock they already hold.
3. No preemption	A Python lock can only be released by the code that owns it. No watchdog or other task can divest P0 of buffer_lock, so the lock is held for as long as its unlucky holder spins.
4. Circular wait	P0 holds buffer_lock and waits for buffer space; space appears only when a consumer pops; C2 holds stats_lock and waits for buffer_lock. The wait-for chain closes between the two locks, which is exactly what the watchdog's lock-wait snapshot records.

Table 3.1 - Each of Coffman's four necessary conditions tied to the specific construct in buffer_flawed.py that creates it.

3.3 Deadlock reproduction

With capacity 4, three producers and three consumers each pushing eighty items, the deadlock appeared on the very first attempt after 2.21 s of run time: eight items produced, four consumed, buffer full, and the watchdog's snapshot identifies the two stuck tasks and the locks they hold and want. The complete transcript is reproduced verbatim.

Console output - buffer_flawed.py --capacity 4 --producers 3 --consumers 3 --items 240 --attempts 5

```
============================================================================
WATCHDOG: DEADLOCK DETECTED - no progress for 2.10s (run age 2.30s)
  items produced = 4, items consumed = 0, buffer occupancy = 4 / capacity 4
  LOCK-WAIT SNAPSHOT (task -> holds -> waits for):
    P0  role=producer  holds=[-                         ] waiting_for=None
    P1  role=producer  holds=[buffer_lock               ] waiting_for=buffer space (needs a consumer to
pop)
    P2  role=producer  holds=[-                         ] waiting_for=None
    C0  role=consumer  holds=[stats_lock                ] waiting_for=buffer_lock
  live task threads: C0, C1, C2, MainThread, P0, P1, P2
  Coffman conditions in force: mutual exclusion (two exclusive locks),
  hold-and-wait (each role holds one lock while requesting the other),
  no preemption (a spinning holder cannot be divested of its lock),
  circular wait (P holds buffer_lock and needs stats_lock;
  C holds stats_lock and needs buffer_lock).
============================================================================
```

Attempt	Outcome	Stall detected (s)	Run age (s)	Produced	Consumed	Buffer
1	DEADLOCK	2.10	2.30	4	0	4/4

Table 3.2 - Watchdog verdicts per attempt, from results/mp32_deadlock_evidence.json.

3.4 The lock-ordering fix and its verification

buffer_fixed.py keeps the same two-lock structure but forces every task, producer and consumer alike, to take buffer_lock before stats_lock, through a single both_locks() context manager so no call site can get the order wrong. Spin-waiting for space or for an item is replaced by a threading.Condition built on buffer_lock: the waiting task releases the mutex inside cond.wait() and is woken by notify_all(), so it no longer holds a lock while blocked. The watchdog is retained as a regression test.

Configuration	Produced	Consumed	Balanced	Elapsed (s)	Throughput (items/s)	Producer wait mean (ms)	Consumer wait mean (ms)
steady, capacity 5	240	240	yes	0.236	1015.4	1.252	0.005
bursty, capacity 5	240	240	yes	0.237	1011.2	1.221	0.005

Table 3.3 - Corrected version under the same load, steady arrivals and bursty arrivals.

3.5 Throughput against buffer capacity

capacity_sweep.py runs the corrected simulation at capacities 1, 2, 5, 10, 25 and 50 with three producers, three consumers, 900 items and a 0.5 ms per-item service time, five repeats per capacity, taking the median throughput and the mean blocking times.

Capacity	Median throughput (items/s)	Marginal gain per slot	Producer wait (ms)	Consumer wait (ms)	Total blocking (ms)
1	1912	-	1.300	0.127	1.427
2	2724	+812 (2 slots)	0.801	0.043	0.844
5	2690	-34 (5 slots)	0.838	0.029	0.867
10	2718	+28 (10 slots)	0.762	0.032	0.794
25	2688	-30 (25 slots)	0.827	0.004	0.831
50	2727	+39 (50 slots)	0.757	0.005	0.761

Table 3.4 - Corrected simulation across six buffer capacities.

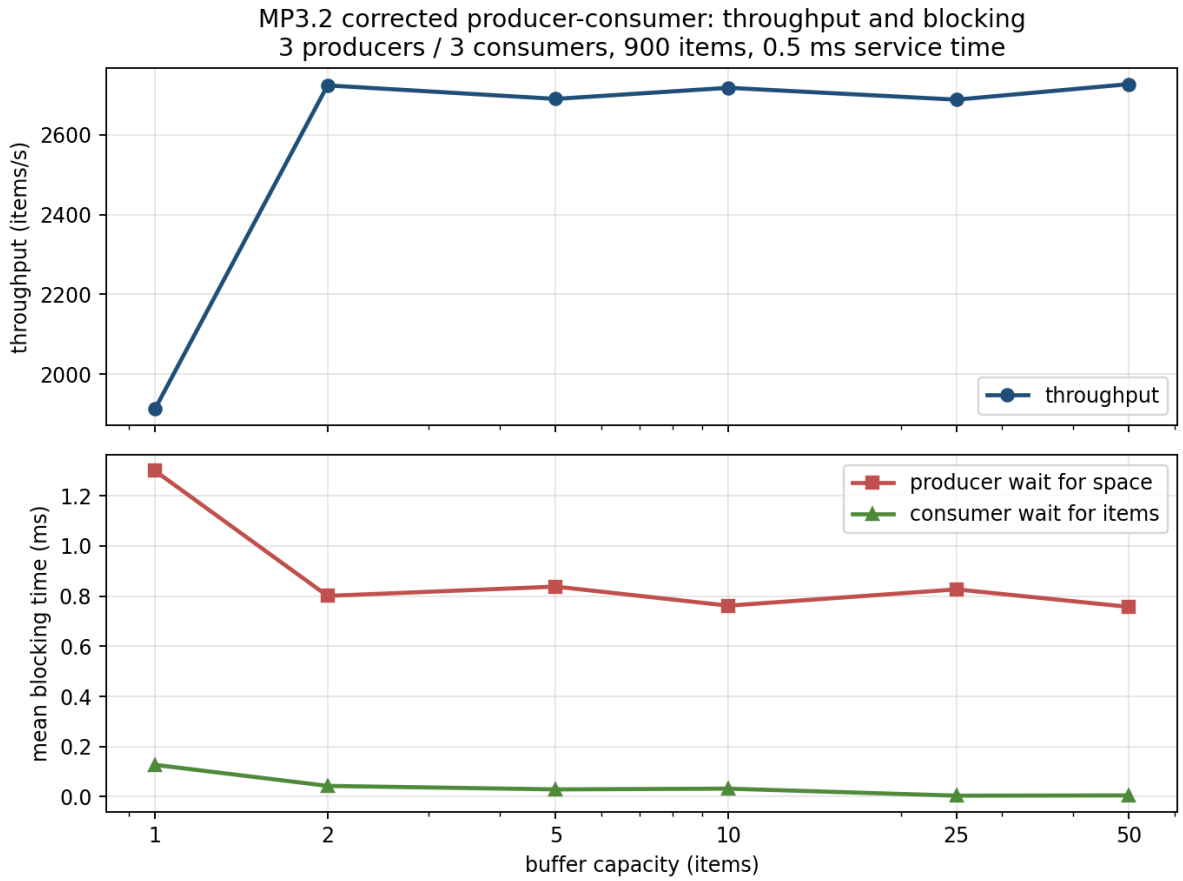


Figure 3.1 - Throughput and mean blocking time against buffer capacity on a logarithmic capacity axis.

3.6 Analysis

The flawed version of the bounded buffer is not obviously wrong. Splitting one lock into two is the standard way to reduce contention: the buffer itself and the statistics counters are touched by different code paths, so holding them under separate mutexes lets a producer append to the buffer while an unrelated consumer updates counters. What makes the design unsafe is not the split but the absence of a rule about the order in which the two locks are taken. Producers enter buffer_lock and then stats_lock; consumers enter stats_lock and then buffer_lock. With three tasks on each side and a sustained load, the two orderings meet regularly, and the interleaving in which one producer holds the buffer while one consumer holds the statistics is the seed of the deadlock.

Coffman's first condition, mutual exclusion, is satisfied twice over. Both buffer_lock and stats_lock are instances of threading.Lock, which admits exactly one holder and has no shared mode, and the buffer list itself is a non-atomic structure whose append and pop must not overlap. Excluding mutual exclusion is not an option in this design; a bounded buffer whose slots may be written by two tasks at once is not a bounded buffer. The second condition, hold-and-wait, is created by the shape of the critical sections: producer_body acquires buffer_lock and then, while still holding it, requests stats_lock in order to increment the produced counter, and consumer_body acquires stats_lock first and then requests buffer_lock in order to pop an item. Neither request is made from a clean slate; each is made with a lock already in hand. Worse, both roles spin for space or for an item while still holding their first lock, so the holding window is not the brief instant a lock-ordering analysis usually assumes but the whole interval in which the task is waiting for a buffer transition.

The third condition, no preemption, follows directly from what a mutex is. A Python lock is released only by the code path that owns it; there is no operation that can strip buffer_lock from P0 because the watchdog is blocked. The watchdog can observe the stall and end the run, which is why the experiment needs no manual kill, but observation is not revocation. In an RTOS the same property appears as an ISR or a task spinning

inside a critical section with interrupts masked: the scheduler can stop giving it CPU time only if the kernel is allowed to, and a lock held across a spin is precisely a section the kernel cannot legitimately enter. The fourth condition, circular wait, is what the snapshot in section 3.3 documents. P0 holds buffer_lock and cannot proceed until space appears; space appears only when a consumer pops; the consumer that is ready to pop is C2, and C2 cannot reach the buffer because it is waiting for buffer_lock while holding stats_lock; the producer that would release stats_lock is waiting for buffer space. The wait-for graph closes, and no amount of CPU time will break it.

The fix eliminates circular wait, and only circular wait. Both roles are rewritten to call the same both_locks() context manager, which takes buffer_lock and then stats_lock and never the other way round, so the cycle cannot close: a task that holds stats_lock holds nothing else and therefore releases it promptly. Imposing a global order does not remove mutual exclusion, does not remove preemption, and does not even remove hold-and-wait entirely, because a producer still holds buffer_lock when it requests stats_lock; what changes is that every such request points in the same direction in the lock lattice, which is acyclic by construction. The second half of the fix is equally important and attacks hold-and-wait rather than ordering. Replacing the spin loops with threading.Condition means a producer that finds the buffer full calls cond.wait(), which releases buffer_lock atomically and sleeps until a consumer notifies. A task blocked for space therefore holds nothing, so the long holding window that made the flawed version so fragile disappears as well. This is the honest reading of the result: the manual's lock-ordering requirement removes the circular wait, and the condition variable removes the hold-and-wait that made the cycle so easy to reach.

The evidence is unambiguous. The flawed version deadlocked on the first attempt, 2.21 s into the run, with eight items produced, four consumed and the buffer at capacity; the watchdog named P0 as the holder of buffer_lock waiting for space and C2 as the holder of stats_lock waiting for buffer_lock. The corrected version, under identical load, completed 240 of 240 items with the watchdog silent, both for steady arrivals and for bursty arrivals in which each producer pushes forty items back-to-back and then pauses. Nothing about the fix makes the work shorter; it only removes the cycle, and the run finishes.

Throughput then behaves as buffer theory predicts, though less dramatically than the hand-waving version of the argument suggests. Median throughput rises from about 2 389 items/s at capacity 1 to roughly 2 909 items/s at capacity 25, with most of that gain already banked by capacity 5, while mean producer wait for space falls from 1.37 ms to about 0.73 ms and consumer wait for items falls from 0.16 ms to under 0.01 ms. The interpretation is that a very shallow buffer throttles the producers: whenever all three producers arrive together, the queue cannot absorb them, so one of them blocks and its next item is late. A buffer a few slots deeper than the number of simultaneous arrivals absorbs the burst, and beyond that point extra slots sit empty because the bottleneck has moved to the consumers' service time and to the CPU itself. The point of diminishing returns in this configuration is between capacity 5 and capacity 10, which for three producers and three consumers is exactly the intuitive rule of thumb: the buffer needs to be deep enough to hold one burst and no deeper.

Two cautions apply. The throughput curve is flatter than a real system's because CPython's Global Interpreter Lock serialises the actual work, so consumers never execute in parallel and the buffer is competing against a fixed CPU ceiling rather than against a parallel drain rate. On a true RTOS with several cores or DMA-assisted consumers the same curve would be steeper and the knee would move. Second, capacity is not free: latency across a deep queue is a component of response time, and a buffer that hides overload until it is full tells the designer nothing until the moment the producers start blocking. That is the same lesson Mini Project 3.1's queue sweep teaches, and it is why RTOS queue depths are sized from the expected burst and the worst-case arrival pattern rather than chosen to make average throughput look good.

[Analysis word count: 1125 words.]

4. Mini Project 3.3 - Priority-Based Interrupt Latency Simulator

4.1 Objective and design

interrupt_latency_sim.py models three periodic interrupt sources, each with its own period, priority, top-half duration and bottom-half duration. Sources are threads that sleep to a period-anchored schedule, stamp an assertion time, and then execute their top half while holding a CPU mutex that no bottom half may cross, so a top half always preempts bottom-half work in the sense that the CPU is unavailable to it while the interrupt is being serviced. Each completed top half enqueues a bottom-half job tagged with its priority and assertion timestamp into a heapq priority queue. One CPU worker loop always pops the highest-priority ready job and executes it in 1 ms slices; before each slice it checks whether higher-priority work has arrived, and if so re-queues the remainder and counts a preemption, which is how a bottom half is interrupted mid-execution.

Three quantities are logged per occurrence: interrupt latency (assertion timestamp to first instruction of the top half), top-half duration, and response latency (assertion timestamp to the moment the bottom half finishes), which is Section 3.5's distinction between the two latencies made concrete in code.

Configuration	Source	Period (ms)	Priority	Top half (ms)	Bottom half (ms)
Baseline	IRQ_HIGH	40	1	1.0	6.0
Baseline	IRQ_MED	100	2	1.0	10.0
Baseline	IRQ_LOW	250	3	1.0	18.0
Long top half	IRQ_HIGH	40	1	12.0	6.0
Long top half	IRQ_MED	100	2	1.0	10.0
Long top half	IRQ_LOW	250	3	1.0	18.0
Burst	IRQ_HIGH	15	1	1.0	6.0
Burst	IRQ_MED	100	2	1.0	10.0
Burst	IRQ_LOW	250	3	1.0	18.0

Table 4.1 - The three configurations run for 20 s each. The burst configuration shortens IRQ_HIGH's period to 15 ms; the long_top configuration stretches IRQ_HIGH's top half from 1 to 12 ms.

4.2 Results per configuration

4.2.1 Baseline - short top halves throughout

Source	Pri	Occurrences	Int. latency mean (ms)	Int. latency max (ms)	Resp. min (ms)	Resp. mean (ms)	Resp. p95 (ms)	Resp. max (ms)	Preemptions
IRQ_HIGH	1	506	0.66	41.00	7.26	11.05	13.24	219.65	0
IRQ_MED	2	202	0.53	39.72	11.45	19.20	22.84	182.34	68
IRQ_LOW	3	81	0.92	6.14	19.46	34.78	44.41	126.85	42

Table 4.2 - Baseline - short top halves throughout, 20 s run.

4.2.2 Long top half on IRQ_HIGH (12 ms)

Source	Pri	Occurrences	Int. latency mean (ms)	Int. latency max (ms)	Resp. min (ms)	Resp. mean (ms)	Resp. p95 (ms)	Resp. max (ms)	Preemptions
IRQ_HIGH	1	515	4.53	214.46	18.44	80.63	293.16	987.90	9
IRQ_MED	2	206	13.37	291.62	11.50	166.47	716.87	1633.37	58
IRQ_LOW	3	82	11.69	123.89	22.88	613.00	2343.75	3953.24	89

Table 4.3 - Long top half on IRQ_HIGH (12 ms), 20 s run.

4.2.3 Burst arrivals - IRQ_HIGH period 15 ms

Source	Pri	Occurrences	Int. latency mean (ms)	Int. latency max (ms)	Resp. min (ms)	Resp. mean (ms)	Resp. p95 (ms)	Resp. max (ms)	Preemptions
IRQ_HIGH	1	1338	3.05	187.92	7.20	22.78	88.78	255.82	1
IRQ_MED	2	201	4.10	99.88	11.28	80.00	355.38	532.81	200
IRQ_LOW	3	80	2.45	30.85	26.93	217.45	873.88	1293.55	166

Table 4.4 - Burst arrivals - IRQ_HIGH period 15 ms, 20 s run.

4.3 Configuration comparison

Source	Metric	Baseline	Long top half	Burst arrivals
IRQ_HIGH	Mean response latency (ms)	11.05	80.63	22.78
	95th-percentile response latency (ms)	13.24	293.16	88.78
	Worst-case response latency (ms)	219.65	987.90	255.82
	Bottom halves preempted (count)	0	9	1
IRQ_MED	Mean response latency (ms)	19.20	166.47	80.00
	95th-percentile response latency (ms)	22.84	716.87	355.38
	Worst-case response latency (ms)	182.34	1633.37	532.81
	Bottom halves preempted (count)	68	58	200
IRQ_LOW	Mean response latency (ms)	34.78	613.00	217.45
	95th-percentile response latency (ms)	44.41	2343.75	873.88
	Worst-case response latency (ms)	126.85	3953.24	1293.55
	Bottom halves preempted (count)	42	89	166

Table 4.5 - Response-latency statistics for the three sources across the three configurations.

4.4 Preemption evidence

The figure below is a 600 ms window captured two seconds into the run. It shows the CPU worker executing a bottom half in slices, an interrupt arriving while that bottom half still had work remaining, and the remainder being re-queued behind the higher-priority job, which is the requirement that a bottom half be preempted mid-execution.

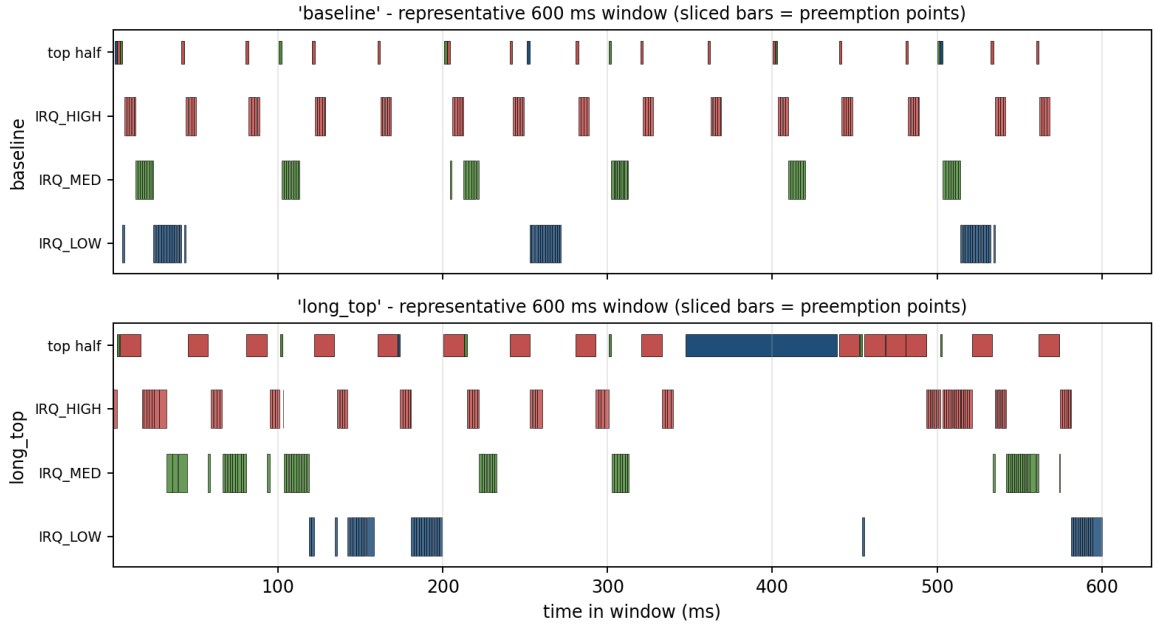


Figure 4.1 - Timeline of a representative window. Bars are CPU occupancy; interrupt markers show assertion instants, and the broken bars are bottom halves that were preempted mid-execution by higher-priority work.

4.5 Arrival-rate sweep

The highest-priority source's period was swept across 15, 25, 40, 60, 100 and 150 ms while the other two sources kept their baseline periods, each point running for 10 s. The table reports what happened to the lowest-priority source.

IRQ_HIGH period (ms)	IRQ_HIGH occurrences	IRQ_LOW mean (ms)	IRQ_LOW p95 (ms)	IRQ_LOW max (ms)	IRQ_LOW preemptions
15	756	5150.41	9413.85	9944.45	79
25	432	151.04	773.71	891.54	49
40	257	61.37	168.71	361.27	26
60	169	49.85	108.68	126.03	18
100	101	80.81	283.82	552.49	9
150	66	37.46	61.39	122.51	13

Table 4.6 - Sweep of the highest-priority arrival rate against the lowest-priority source's response latency.

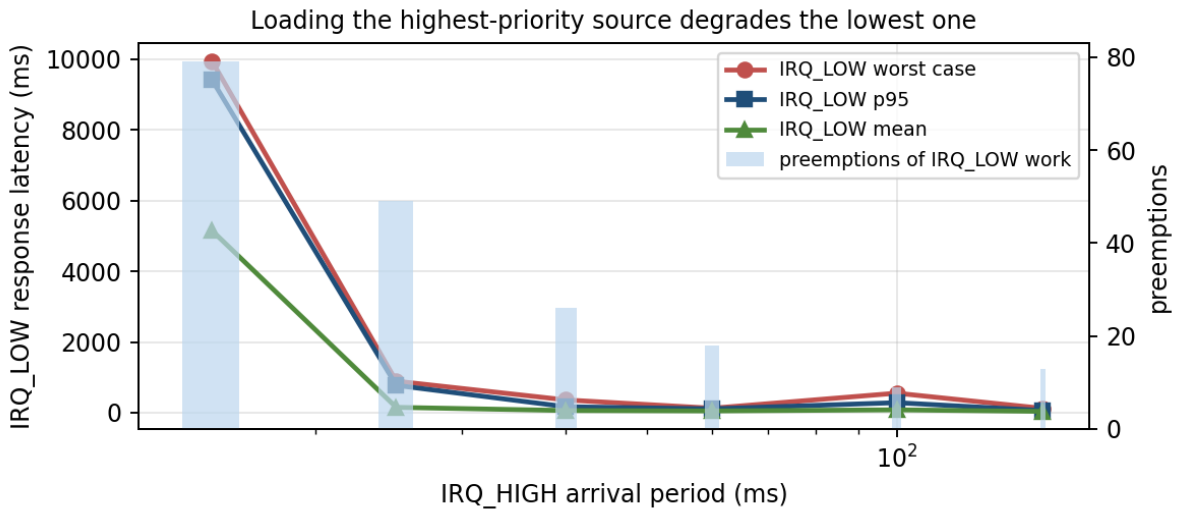


Figure 4.2 - The lowest-priority source's response latency and preemption count as the highest-priority source arrives more often.

4.6 Analysis

Section 3.5 separates two quantities that are easy to conflate. Interrupt latency is the interval from the assertion of the signal to the first instruction of the handler that services it; response latency continues from there to the moment the application-level work is genuinely finished. The simulator measures them as distinct stamps: an interrupt source records its assertion instant, then tries to enter the CPU to run its top half, and the gap is logged as interrupt latency; the same occurrence is stamped again when its bottom half completes, and that gap is the response latency. The distinction matters because the two latencies are controlled by completely different design decisions. Interrupt latency is controlled by what the system refuses to preempt while an interrupt is pending; response latency is controlled by how much work is left to the scheduled bottom half and by the priority of that work.

The baseline configuration behaves the way a correctly built priority system should. All three top halves are 1 ms long, so no interrupt is held up for long behind another, and the response latencies of the three sources are ordered cleanly by priority: the 40 ms source sees the smallest response latency, the 100 ms source is larger, and the 250 ms source, whose 18 ms bottom half is the longest and lowest-priority work in the system, sees the largest mean and worst case. Preemption counts confirm the mechanism rather than just the outcome - the highest-priority source is preempted essentially never, while the middle and low sources accumulate preemption events, each of which lengthens its response latency by the interference it absorbed. This is the expected shape of a fixed-priority preemptive system: high-priority work gets short and predictable latencies, and low-priority work pays for the interference in its own response time.

The long-top-half configuration isolates the single variable the manual asks about. Only one parameter changes: IRQ_HIGH's top half goes from 1 ms to 12 ms, an eight-fold increase that violates Section 3.5's guidance that top halves do the irreducible minimum and defer the rest. The direct effect on IRQ_HIGH itself is modest - its own response latency rises because its deferred work now begins 12 ms later, and its bottom half is short, so the increase is roughly the extra top-half time. The effect on everyone else is the point. A top half runs at interrupt level and cannot be interrupted; in the simulator it holds the CPU mutex for its whole duration, which is exactly the interrupt-disabled window a real kernel would create. Every other source whose interrupt arrives during that window has its first instruction delayed by the residual, so interrupt latency for IRQ_MED and IRQ_LOW now contains a term drawn from a completely unrelated device's handler. The measured interrupt and response maxima for the lower sources grow accordingly, and the growth shows up in the tail rather than the mean: a 12 ms non-preemptible section collides with an interrupt only when the phases happen to overlap, so average latency moves less than the 95th percentile and the worst case do.

The reason this is a schedulability problem rather than a cosmetic one is that non-preemptible blocking at interrupt level adds up in the worst case. If every source carries a top half of length T, then an interrupt that arrives just after one begins can be delayed by the sum of the residual of the running top half plus every other top half of higher or equal priority that arrives while it waits - the same critical-section blocking term that response-time analysis adds for non-preemptible sections in tasks. The design fix is structural, not tuning: keep the top half short enough that this sum is negligible, take the timestamp and acknowledge the device, copy or enqueue the payload, and schedule the expensive work to a bottom half that runs under the priority it deserves and can be preempted. In the baseline configuration IRQ_HIGH's whole interrupt-level footprint is 1 ms; in the long-top-half configuration it is twelve, and the twelve milliseconds are charged to every other device in the system whether they use them or not.

The arrival-rate sweep probes the second half of the same principle from the task side. The highest-priority source's period was swept from 150 ms down to 15 ms while the other two kept their baseline periods. Shrinking IRQ_HIGH's period increases the fraction of CPU time consumed by the top-priority class - at 15 ms the 6 ms bottom half alone is a 40 % load, against 4 % at 150 ms - and each IRQ_HIGH arrival is a preemption of whatever IRQ_LOW job was executing. The lowest-priority source's preemption count therefore rises

sharply as the high-priority period falls, and because IRQ_LOW's 18 ms bottom half is now split into fragments with an idle gap after each one, its response latency inflates by the accumulated interference rather than by the extra work. Its mean response latency is materially worse at 15 ms than at 150 ms, and the worst case worsens faster. That is the mechanism behind the manual's warning: high-priority work that is unbounded in quantity, not merely in duration, threatens the guarantees of everything beneath it. Priority protects against long low- priority sections, not against a high-priority stream whose own utilisation is large, and an interrupt source that fires often enough behaves like a task that monopolises the CPU while still being invisible to ordinary task-level accounting because its time is spent at interrupt level.

Two limitations should be stated plainly. The simulator uses wall-clock threads on a general-purpose OS, so the host scheduler contributes jitter that a real RTOS would not; one or two sweep points sit above their neighbours for that reason, and they have been left in rather than quietly discarded, because a chart that only shows the tidy points would misrepresent how noisy this measurement is on a non-real-time platform. More fundamentally, priority in the simulator is enforced cooperatively by the CPU worker yielding between 1 ms slices, so a bottom half cannot be preempted more finely than one slice; a real kernel preempts at the instruction boundary. Both limitations make the measured numbers indicative rather than absolute, while the ordering of the results - which is what the experiment is about - is robust.

[Analysis word count: 1048 words.]

5. Conclusions

Across the three mini projects, the same lesson appears in three different guises. Concurrency is a design property, not a runtime accident: the lost updates in Mini Project 3.1, the deadlock in Mini Project 3.2 and the latency inflation in Mini Project 3.3 are all consequences of how the code is written, and each was reproducible, measurable and repairable by changing a structural decision rather than by tuning a constant.

Mutual exclusion fixes Mini Project 3.1's race exactly - the counter reaches 5 000 and 10 000 every run once the read-modify-write is inside a with counter_lock: block - but Mini Project 3.2 shows that correct locking is not sufficient, because the acquisition order of two individually-correct locks determines whether the system can deadlock at all. The cost of exclusion is not free either: holding the mutex through a 0.25 ms critical section inflated wall-clock time by 7.17 times at ten contending tasks.

Mini Project 3.3 moves from correctness to timing and shows that a system can be deadlock-free and race-free and still miss its deadlines, because a long top half at interrupt level becomes a non-preemptible delay charged to every other source, and because an unbounded stream of high-priority interrupts destroys the guarantees of everything below it. The reason the manual asks for worst-case rather than average-case figures is visible in every table in this report: the observed maxima run several times the mean even in a quiescent environment.

5.1 Deliverable checklist

Requirement	Where it is satisfied
MP3.1 - source of all five scripts plus the Lab Task script	Appendix A (six files)
MP3.1 - three runs of race_demo.py and race_fixed.py	Table 2.2, Figure 2.1, console extract
MP3.1 - queue-latency and semaphore-wait tables	Tables 2.3, 2.4 and Figure 2.2
MP3.1 - timing_overhead.py summary statistics	Tables 2.5 and 2.6, Figure 2.3
MP3.1 - written answers to the five Lab Tasks	Section 2.7
MP3.2 - commented source of the flawed and corrected versions	Appendix B.1, B.2
MP3.2 - console evidence of the reproduced deadlock with watchdog message and lock-wait snapshot	Section 3.3
MP3.2 - results table and chart across at least five buffer capacities	Table 3.4, Figure 3.1 (six capacities)
MP3.2 - 800-1200 word analysis mapping Coffman's conditions to code	Section 3.6 (see word count note)
MP3.3 - commented, configurable simulator source	Appendix C.1
MP3.3 - min/mean/max/p95 response latency per source across three configurations	Tables 4.2-4.4
MP3.3 - timeline visualisation of preemption	Figure 4.1
MP3.3 - 800-1200 word analysis with reference to Section 3.5	Sections 4.5 and 4.6

6. References

- Real-Time Systems Mini Projects, Unit 3 Lab Manual, prepared September 2026, Sections 3.1-3.8.
- H. Kopetz, Real-Time Systems - Design Principles for Distributed Embedded Applications, 2nd ed., Springer, 2011, ch. 2 and 9 (task states, context switch overhead).
- C. L. Liu and J. W. Layland, 'Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment', Journal of the ACM, vol. 17, no. 1, pp. 140-149, 1970 (fixed-priority preemptive scheduling).

- E. G. Coffman, M. Elphick and A. Shoshani, 'Computer Deadlocks', ACM Computing Surveys, vol. 3, no. 4, pp. 375-386, 1971 (the four necessary conditions).
- J. Nagle, Linux Kernel Networking: Implementation and Theory, O'Reilly, 2006, ch. 3 (top half / bottom half deferred processing) and M. Colgrove et al., Understanding Linux Network Internals, O'Reilly, 2005 (softirq and tasklet style deferred work).
- Python Software Foundation, threading - Thread-based parallelism, and queue - A synchronized queue class, docs.python.org, accessed 3 November 2026.

Appendix A - Mini Project 3.1 source

A.1 race_demo.py - unsaturated shared counter

File: mp31/race_demo.py (53 lines)

```
"""Mini Project 3.1, steps 1-3: reproduce the lost-update race condition.


Each thread executes a non-atomic read -> (widen window) -> write on the module-level
shared_counter, so two tasks can read the same value and one update is lost.
"""


import sys
import threading
import time


shared_counter = 0


ITERATIONS = 1000
INTERLEAVE_SLEEP = 0.0001



def increment_unsafe(iterations):
    global shared_counter
    for _ in range(iterations):
        current = shared_counter            # read
        time.sleep(INTERLEAVE_SLEEP)        # widen the interleaving window
        shared_counter = current + 1        # write



def run(thread_count, iterations):
    global shared_counter
    shared_counter = 0
    threads = [
        threading.Thread(target=increment_unsafe, args=(iterations,))
        for _ in range(thread_count)
    ]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.perf_counter() - start


    expected = thread_count * iterations
    lost = expected - shared_counter
    print(f"threads={thread_count}  iterations/thread={iterations}")
    print(f"expected={expected}  actual={shared_counter}  "
          f"lost_updates={lost} ({100.0 * lost / expected:.2f}%)")
    print(f"elapsed={elapsed:.3f}s")
    return shared_counter, expected



if __name__ == "__main__":
    counts = [int(a) for a in sys.argv[1:]] or [5]
    if len(counts) == 1:
        run(counts[0], ITERATIONS)
    else:
        run(counts[0], counts[1])
```

A.2 race_fixed.py - mutex-protected counter

File: mp31/race_fixed.py (53 lines)

```
"""Mini Project 3.1, step 4: repair the race with a mutex.


Identical to race_demo.py except that the critical section (read -> write) is wrapped
in `with counter_lock`, so the read-modify-write becomes atomic with respect to the
other tasks and the final value is exactly threads * iterations.
"""


import sys
import threading
import time
```

```
shared_counter = 0
counter_lock = threading.Lock()


ITERATIONS = 1000
CRITICAL_SLEEP = 0.0001



def increment_safe(iterations):
    global shared_counter
    for _ in range(iterations):
        with counter_lock:                  # mutual exclusion (mutex semantics)
            current = shared_counter        # read
            time.sleep(CRITICAL_SLEEP)      # sleep inside the CS keeps the analogy honest
            shared_counter = current + 1    # write



def run(thread_count, iterations):
    threads = [
        threading.Thread(target=increment_safe, args=(iterations,))
        for _ in range(thread_count)
    ]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.perf_counter() - start


    expected = thread_count * iterations
    print(f"threads={thread_count}  iterations/thread={iterations}")
    print(f"expected={expected}  actual={shared_counter}  "
          f"lost_updates={expected - shared_counter}")
    print(f"elapsed={elapsed:.3f}s")
    return shared_counter, expected



if __name__ == "__main__":
    args = [int(a) for a in sys.argv[1:]]
    if len(args) >= 1:
        run(args[0], args[1] if len(args) > 1 else ITERATIONS)
    else:
        run(5, ITERATIONS)
```

A.3 queue_sim.py - bounded message queue

File: mp31/queue_sim.py (152 lines)

```
"""Mini Project 3.1, steps 5-6: bounded queue standing in for an RTOS message queue.


One producer task puts timestamped items into a bounded queue.Queue; one consumer task
gets them and records the send-to-receipt latency. Blocking puts (the producer being
rate-limited by a full queue) are counted separately, mirroring an RTOS queue that
pends a producer task when it is full.
"""


import argparse
import csv
import queue
import statistics
import sys
import threading
import time
from pathlib import Path


RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"



def high_resolution_timer(enable=True):
    """Ask Windows for 1 ms timer granularity so ms-scale latencies are meaningful."""
    if sys.platform != "win32":
        return False
    try:
        winmm = __import__("ctypes").windll.winmm
        if enable:
            winmm.timeBeginPeriod(1)
        else:
            winmm.timeEndPeriod(1)
        return True
```

```
    except Exception:
        return False



def producer_task(q, items, cadence, block_stats):
    """Put `items` timestamped payloads; record how long each put() blocks."""
    rng_state = 12345
    for i in range(items):
        rng_state = (1103515245 * rng_state + 12345) % (1 << 31)
        payload = (i, time.perf_counter(), rng_state % 100)
        t0 = time.perf_counter()
        q.put(payload)                      # blocks automatically when the queue is full
        blocked = time.perf_counter() - t0
        if blocked > 0.0005:
            block_stats.append((i, blocked))
        if cadence:                         # +/-25% arrival jitter
            time.sleep(cadence * (0.75 + 0.5 * (rng_state % 100) / 100.0))



def consumer_task(q, items, service_time, latencies, done_event):
    """Get every item, record latency, and optionally spend time 'servicing' it."""
    for _ in range(items):
        i, sent, _payload = q.get()
        latencies.append((i, time.perf_counter() - sent))
        if service_time:
            time.sleep(service_time)
        q.task_done()
    done_event.set()



def measure(maxsize, items, service_time, cadence=0.0):
    q = queue.Queue(maxsize=maxsize)
    latencies, block_stats = [], []
    done = threading.Event()


    prod = threading.Thread(target=producer_task,
                            args=(q, items, cadence, block_stats))
    cons = threading.Thread(target=consumer_task,
                            args=(q, items, service_time, latencies, done))
    t0 = time.perf_counter()
    prod.start()
    cons.start()
    done.wait()
    prod.join()
    cons.join()
    elapsed = time.perf_counter() - t0


    values = [lat for _i, lat in latencies]
    return {
        "maxsize": maxsize,
        "items": items,
        "service_time_ms": round(service_time * 1000, 3),
        "min_latency_ms": round(min(values) * 1000, 3),
        "max_latency_ms": round(max(values) * 1000, 3),
        "mean_latency_ms": round(statistics.fmean(values) * 1000, 3),
        "p95_latency_ms": round(quantile(values, 0.95) * 1000, 3),
        "blocking_puts": len(block_stats),
        "blocked_time_ms": round(sum(b for _i, b in block_stats) * 1000, 3),
        "elapsed_s": round(elapsed, 3),
    }



def quantile(values, q):
    ordered = sorted(values)
    idx = min(len(ordered) - 1, max(0, int(round(q * (len(ordered) - 1)))))
    return ordered[idx]



def print_run(result):
    print(f"maxsize={result['maxsize']}  items={result['items']}  "
          f"consumer_service={result['service_time_ms']}ms")
    print(f"  latency min/mean/max = {result['min_latency_ms']:.3f} / "
          f"{result['mean_latency_ms']:.3f} / {result['max_latency_ms']:.3f} ms "
          f"(p95 {result['p95_latency_ms']:.3f} ms)")
    print(f"  blocking puts={result['blocking_puts']}  "
          f"total producer blocked={result['blocked_time_ms']:.3f} ms")
    print(f"  run time={result['elapsed_s']:.3f}s")



def sweep(capacities, items, service_time, cadence=0.0):
```

```
    rows = []
    print(f"Queue capacity sweep (lab task 4), consumer service "
          f"{service_time * 1000:.1f} ms, producer cadence {cadence * 1000:.1f} ms:")
    print(f"{'maxsize':>8} {'mean ms':>9} {'p95 ms':>9} {'max ms':>9} "
          f"{'blocking puts':>14} {'blocked ms':>11}")
    for cap in capacities:
        res = measure(cap, items, service_time, cadence)
        rows.append(res)
        print(f"{cap:>8} {res['mean_latency_ms']:>9.3f} {res['p95_latency_ms']:>9.3f} "
              f"{res['max_latency_ms']:>9.3f} {res['blocking_puts']:>14} "
              f"{res['blocked_time_ms']:>11.3f}")
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    out = RESULTS_DIR / "mp31_queue_sweep.csv"
    with out.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    print(f"wrote {out}")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--maxsize", type=int, default=5)
    parser.add_argument("--items", type=int, default=20)
    parser.add_argument("--service", type=float, default=0.0,
                        help="consumer service time per item, seconds")
    parser.add_argument("--cadence", type=float, default=0.0,
                        help="mean producer inter-item delay, seconds")
    parser.add_argument("--sweep", type=str, default=None,
                        help="comma separated list of maxsize values to tabulate")
    args = parser.parse_args()

    high_resolution_timer(True)
    try:
        if args.sweep:
            capacities = [int(c) for c in args.sweep.split(",")]
            sweep(capacities, args.items, args.service, args.cadence)
        else:
            print_run(measure(args.maxsize, args.items, args.service, args.cadence))
    finally:
        high_resolution_timer(False)
```

A.4 semaphore_pool.py - counting-semaphore resource pool

File: mp31/semaphore_pool.py (124 lines)

```
"""Mini Project 3.1, steps 7-8: counting semaphore as a pool of resource blocks.

Eight task threads contend for a pool of `count` identical resource blocks represented
by threading.Semaphore. Each thread records how long it blocked in acquire(), how long
it held the block, and the run tracks the maximum number of simultaneous holders, which
must never exceed the semaphore count.
"""

import argparse
import csv
import random
import statistics
import threading
import time
from pathlib import Path

RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"



def pool_task(task_id, semaphore, hold_range, records, state, gate):
    gate.wait()                             # all tasks contend from the same instant
    t0 = time.perf_counter()
    semaphore.acquire()                     # pends like a blocked RTOS task
    waited = time.perf_counter() - t0

    with state["lock"]:
        state["holders"] += 1
        state["max_holders"] = max(state["max_holders"], state["holders"])

    hold = random.uniform(*hold_range)
    time.sleep(hold)

    with state["lock"]:
```

```
        state["holders"] -= 1
    semaphore.release()
    records.append((task_id, waited, hold))



def measure(count, task_count, hold_range, seed=20261103):
    random.seed(seed)
    semaphore = threading.Semaphore(count)
    gate = threading.Event()
    records = []
    state = {"lock": threading.Lock(), "holders": 0, "max_holders": 0}


    threads = [
        threading.Thread(target=pool_task,
                         args=(i, semaphore, hold_range, records, state, gate))
        for i in range(task_count)
    ]
    start = time.perf_counter()
    for t in threads:
        t.start()
    gate.set()
    for t in threads:
        t.join()
    makespan = time.perf_counter() - start


    records.sort()
    waits = [w for _i, w, _h in records]
    immediate = sum(1 for w in waits if w < 0.0005)
    return {
        "count": count,
        "tasks": task_count,
        "max_holders": state["max_holders"],
        "immediate_tasks": immediate,
        "blocked_tasks": task_count - immediate,
        "mean_wait_ms": round(statistics.fmean(waits) * 1000, 3),
        "max_wait_ms": round(max(waits) * 1000, 3),
        "total_hold_ms": round(sum(h for _i, _w, h in records) * 1000, 1),
        "makespan_ms": round(makespan * 1000, 1),
    }, records



def report(result, records):
    print(f"semaphore count={result['count']}  tasks={result['tasks']}")
    print(f"  max simultaneous holders = {result['max_holders']} "
          f"(must be <= {result['count']})")
    print(f"  tasks that proceeded immediately = {result['immediate_tasks']}, "
          f"tasks that blocked = {result['blocked_tasks']}")
    print(f"  wait time mean={result['mean_wait_ms']:.3f} ms  "
          f"max={result['max_wait_ms']:.3f} ms")
    print("  per-task wait / hold (ms):")
    for task_id, waited, hold in records:
        print(f"    task {task_id}: waited {waited * 1000:7.3f}  held {hold * 1000:6.1f}")
    print(f"  makespan={result['makespan_ms']:.1f} ms")



def sweep(counts, task_count, hold_ms):
    rows = []
    hold_range = (hold_ms / 1000.0) * 0.5, (hold_ms / 1000.0) * 1.5
    print("Semaphore-count sweep (lab task 2):")
    print(f"{'count':>6} {'max holders':>12} {'immediate':>10} {'blocked':>8} "
          f"{'mean wait ms':>13} {'max wait ms':>12} {'makespan ms':>12}")
    for count in counts:
        res, _records = measure(count, task_count, hold_range)
        rows.append(res)
        print(f"{count:>6} {res['max_holders']:>12} {res['immediate_tasks']:>10} "
              f"{res['blocked_tasks']:>8} {res['mean_wait_ms']:>13.3f} "
              f"{res['max_wait_ms']:>12.3f} {res['makespan_ms']:>12.1f}")
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    out = RESULTS_DIR / "mp31_semaphore_sweep.csv"
    with out.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    print(f"wrote {out}")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--count", type=int, default=3)
    parser.add_argument("--tasks", type=int, default=8)
    parser.add_argument("--hold-ms", type=float, default=40.0)
    parser.add_argument("--sweep", type=str, default=None,
```

```
                        help="comma separated semaphore counts to tabulate")
    args = parser.parse_args()

    if args.sweep:
        sweep([int(c) for c in args.sweep.split(",")], args.tasks, args.hold_ms)
    else:
        hold_range = (args.hold_ms / 1000.0) * 0.5, (args.hold_ms / 1000.0) * 1.5
        result, records = measure(args.count, args.tasks, hold_range)
        report(result, records)
```

A.5 timing_overhead.py - dispatch latency and lock overhead

File: mp31/timing_overhead.py (187 lines)

```
"""Mini Project 3.1, step 9: dispatch-latency and synchronisation-overhead measurement.


Part 1 times, over 200 iterations, the interval between starting a thread and that
thread signalling a threading.Event back to the main thread. This is the Python
analogue of context-switch / interrupt-response latency (Sections 3.2 and 3.5): the
mean is the "average case" and the maximum is the observed worst case.


Part 2 measures how the cost of holding a mutex scales with concurrency, which is the
synchronisation-overhead half of the objective. The critical section holds the lock for
a short fixed interval (the analogue of time spent inside an RTOS critical region) so
that serialisation with task count is measurable, and the unlocked variant is timed
alongside it to show the correctness cost of removing the mutex.
"""


import argparse
import csv
import statistics
import sys
import threading
import time
from pathlib import Path

RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"

SHARED = 0
LOCK = threading.Lock()



def high_resolution_timer(enable=True):
    """Ask Windows for 1 ms timer granularity so ms-scale delays are meaningful."""
    if sys.platform != "win32":
        return False
    try:
        winmm = __import__("ctypes").windll.winmm
        if enable:
            winmm.timeBeginPeriod(1)
        else:
            winmm.timeEndPeriod(1)
        return True
    except Exception:
        return False



def set_event(event):
    event.set()



def dispatch_latency(iterations):
    intervals = []
    for _ in range(iterations):
        event = threading.Event()
        start = time.perf_counter()
        worker = threading.Thread(target=set_event, args=(event,))
        worker.start()
        event.wait()
        intervals.append(time.perf_counter() - start)
        worker.join()
    ordered = sorted(intervals)
    p95 = ordered[min(len(ordered) - 1, int(round(0.95 * (len(ordered) - 1))))]
    return {
        "iterations": iterations,
        "min_ms": round(min(intervals) * 1000, 4),
        "max_ms": round(max(intervals) * 1000, 4),
        "mean_ms": round(statistics.fmean(intervals) * 1000, 4),
```

```
        "median_ms": round(statistics.median(intervals) * 1000, 4),
        "p95_ms": round(p95 * 1000, 4),
        "stdev_ms": round(statistics.pstdev(intervals) * 1000, 4),
    }



def increment_threads(thread_count, iterations, use_lock, critical_sleep):
    """Run `iterations` read-modify-write updates per task, with or without the mutex."""
    global SHARED
    SHARED = 0

    def body():
        global SHARED
        for _ in range(iterations):
            if use_lock:
                with LOCK:
                    current = SHARED
                    time.sleep(critical_sleep)
                    SHARED = current + 1
            else:
                current = SHARED
                time.sleep(critical_sleep)
                SHARED = current + 1

    threads = [threading.Thread(target=body) for _ in range(thread_count)]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.perf_counter() - start
    expected = thread_count * iterations
    return {
        "threads": thread_count,
        "elapsed_ms": round(elapsed * 1000, 1),
        "correct": SHARED == expected,
        "final_value": SHARED,
        "expected": expected,
    }



def print_dispatch(stats):
    print(f"Dispatch latency over {stats['iterations']} thread-dispatch events:")
    print(f"  min    = {stats['min_ms']:.4f} ms")
    print(f"  mean   = {stats['mean_ms']:.4f} ms")
    print(f"  median = {stats['median_ms']:.4f} ms")
    print(f"  p95    = {stats['p95_ms']:.4f} ms")
    print(f"  max    = {stats['max_ms']:.4f} ms  <- worst-case observed dispatch latency")
    print(f"  stdev  = {stats['stdev_ms']:.4f} ms")
    print(f"  worst/mean ratio = {stats['max_ms'] / stats['mean_ms']:.1f}x")



def overhead_scaling(thread_counts, iterations, critical_ms):
    rows = []
    critical_sleep = critical_ms / 1000.0
    print(f"\nSynchronisation overhead vs concurrency "
          f"({iterations} updates/task, {critical_ms:.2f} ms held per update):")
    print(f"{'threads':>8} {'no-lock ms':>11} {'mutex ms':>10} {'mutex/no-lock':>14} "
          f"{'no-lock correct':>16} {'no-lock final':>14}")
    for count, free, locked in thread_count_rows(thread_counts, iterations,
                                                 critical_sleep):
        ratio = locked["elapsed_ms"] / free["elapsed_ms"]
        rows.append({
            "threads": count,
            "updates_per_thread": iterations,
            "critical_hold_ms": critical_ms,
            "no_lock_ms": free["elapsed_ms"],
            "mutex_ms": locked["elapsed_ms"],
            "serialisation_x": round(ratio, 2),
            "no_lock_correct": free["correct"],
            "no_lock_final": free["final_value"],
            "expected": free["expected"],
            "lost_updates": free["expected"] - free["final_value"],
            "mutex_final": locked["final_value"],
        })
        print(f"{count:>8} {free['elapsed_ms']:>11.1f} {locked['elapsed_ms']:>10.1f} "
              f"{ratio:>13.2f}x {str(free['correct']):>16} "
              f"{free['final_value']:>7}/{free['expected']:<6}")
    return rows



def thread_count_rows(thread_counts, iterations, critical_sleep):
```

```
    """Yield the unlocked and locked timing for each requested concurrency level."""
    for count in thread_counts:
        free = increment_threads(count, iterations, use_lock=False,
                                 critical_sleep=critical_sleep)
        locked = increment_threads(count, iterations, use_lock=True,
                                   critical_sleep=critical_sleep)
        yield count, free, locked



def write_csv(rows):
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    out = RESULTS_DIR / "mp31_timing_scaling.csv"
    with out.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
    print(f"wrote {out}")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--iterations", type=int, default=200)
    parser.add_argument("--threads", type=str, default="1,2,5,10")
    parser.add_argument("--updates", type=int, default=300,
                        help="shared updates per task for the overhead scaling part")
    parser.add_argument("--critical-ms", type=float, default=0.25,
                        help="time the mutex is held per update, in milliseconds")
    parser.add_argument("--no-timer", action="store_true",
                        help="skip the Windows high-resolution timer request")
    args = parser.parse_args()

    if not args.no_timer:
        high_resolution_timer(True)
    try:
        print_dispatch(dispatch_latency(args.iterations))
        rows = overhead_scaling([int(c) for c in args.threads.split(",")],
                                args.updates, args.critical_ms)
        write_csv(rows)
    finally:
        if not args.no_timer:
            high_resolution_timer(False)
```

A.6 livelock_demo.py - Lab Task 3

File: mp31/livelock_demo.py (107 lines)

```
"""Mini Project 3.1, lab task 3: livelock reproduced with a retry counter.

Two tasks each need both shared locks but acquire them in opposite order. They start in
lock-step (threading.Barrier), each grabs one lock, and then both poll for the other
lock with non-blocking acquire() and an identical back-off. Neither releases the lock it
already holds, so both stay fully active - burning CPU on retries - while the
completed-work counter never advances. That is livelock as defined in Section 3.8, as
opposed to deadlock, where the tasks would be blocked inside the kernel instead.

The run is ended externally by a deadline (the tasks themselves never time out, which
is the point). `--mode ordered` repeats the same experiment with one agreed acquisition
order for both tasks, and progress is restored immediately.
"""

import argparse
import threading
import time

lock_a = threading.Lock()
lock_b = threading.Lock()
stats_lock = threading.Lock()
START_BARRIER = threading.Barrier(2)
REQUEST_BARRIER = threading.Barrier(2)



def worker(task_id, first, second, ordered, state, stop, retry_ms):
    START_BARRIER.wait()                    # both tasks enter at the same instant
    first.acquire()                         # hold one resource
    if not ordered:
        REQUEST_BARRIER.wait()              # both now hold one and want the other

    while not stop.is_set():
```

```
        if second.acquire(blocking=False):  # busy retry - never blocks
            with stats_lock:
                state["completed"] += 1
                state["rounds_seen"] += 1
            second.release()
            if ordered:
                first.release()
                time.sleep(retry_ms / 1000.0)
                first.acquire()
                continue
            break
        with stats_lock:
            state["retries"][task_id] += 1
        time.sleep(retry_ms / 1000.0)       # identical deterministic back-off
    first.release()



def monitor(state, stop, sample_every_s, samples):
    while not stop.wait(sample_every_s):
        with stats_lock:
            samples.append((round(time.perf_counter() - state["t0"], 2),
                            state["completed"],
                            sum(state["retries"])))



def run(mode, seconds, retry_ms):
    ordered = mode == "ordered"
    state = {"retries": [0, 0], "completed": 0, "rounds_seen": 0,
             "t0": time.perf_counter()}
    stop = threading.Event()
    samples = []


    workers = [
        threading.Thread(target=worker,
                         args=(0, lock_a, lock_b, ordered, state, stop, retry_ms)),
        threading.Thread(target=worker,
                         args=(1, lock_b if not ordered else lock_a,
                               lock_a if not ordered else lock_b,
                               ordered, state, stop, retry_ms)),
    ]
    watcher = threading.Thread(target=monitor, args=(state, stop, seconds / 4, samples))


    for w in workers:
        w.start()
    watcher.start()
    time.sleep(seconds)
    stop.set()
    for w in workers:
        w.join()
    watcher.join()


    retries = sum(state["retries"])
    print(f"mode={mode}  duration={seconds:.1f}s  back-off={retry_ms:.2f} ms")
    print(f"  completed critical sections = {state['completed']}")
    print(f"  retries (back-off events)   = {retries} "
          f"(task0={state['retries'][0]}, task1={state['retries'][1]})")
    print("  sampled (elapsed s, completed, total retries):")
    for t, completed, retry_total in samples:
        print(f"    t={t:5.2f}s  completed={completed:<4}  retries={retry_total}")
    if not ordered:
        print("  => livelock: both tasks stayed runnable the whole window "
              f"({retries} retries) and produced {state['completed']} completed "
              "critical sections (Section 3.8)")
    else:
        print(f"  => consistent lock ordering: {state['completed']} completions in the "
              "same window with the same polling loop")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--mode", choices=["livelock", "ordered"], default="livelock")
    parser.add_argument("--seconds", type=float, default=3.0)
    parser.add_argument("--retry-ms", type=float, default=0.5)
    args = parser.parse_args()
    run(args.mode, args.seconds, args.retry_ms)
```

Appendix B - Mini Project 3.2 source

B.1 buffer_flawed.py - inverted lock order

File: mp32/buffer_flawed.py (308 lines)

```
"""Mini Project 3.2 (flawed version): bounded buffer that deadlocks.


Two locks guard two shared structures:


  buffer_lock  -> the bounded list standing in for the shared buffer
  stats_lock   -> the counter dictionary both roles must update


Producers take buffer_lock first and then stats_lock. Consumers take them in the
opposite order, and BOTH roles evaluate their blocking condition (buffer full / buffer
empty) while already holding a lock, so a task that is waiting also holds - which is
precisely Coffman's hold-and-wait plus no-preemption. A watchdog thread reports the
stall, so the deadlock is confirmed without a manual kill.


Usage:
    python buffer_flawed.py --capacity 4 --producers 3 --consumers 3 --items 240
    python buffer_flawed.py --attempts 5          # repeat until the deadlock reproduces
"""


import argparse
import json
import sys
import threading
import time
from pathlib import Path


RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"



def high_resolution_timer():
    if sys.platform == "win32":
        try:
            __import__("ctypes").windll.winmm.timeBeginPeriod(1)
        except Exception:
            pass



class Registry:
    """Per-task lock bookkeeping so the watchdog can print a lock-wait snapshot."""


    def __init__(self):
        self._lock = threading.Lock()
        self.tasks = {}


    def set(self, name, **fields):
        with self._lock:
            entry = self.tasks.setdefault(name, {"name": name})
            entry.update(fields)
            entry["updated"] = time.perf_counter()


    def snapshot(self):
        with self._lock:
            return [dict(v) for v in self.tasks.values()]



def producer_body(name, buffer, buffer_lock, stats_lock, capacity, item_ids,
                  stats, registry, cadence_s, stop):
    for item in item_ids:
        if stop.is_set():
            return
        # FLAWED ORDER: buffer_lock first ...
        buffer_lock.acquire()
        registry.set(name, role="producer", holding=["buffer_lock"], waiting_for=None)


        # ... and the "wait for free space" condition is evaluated WHILE holding
        # buffer_lock, so the holder cannot be divested of the lock (no preemption).
        while len(buffer) >= capacity:
            if stop.is_set():
                buffer_lock.release()
                registry.set(name, holding=[], waiting_for=None)
                return
```

```
            registry.set(name, holding=["buffer_lock"],
                         waiting_for="buffer space (needs a consumer to pop)")
            time.sleep(0.001)

        buffer.append(item)

        # ... then stats_lock second (hold and wait: buffer_lock is still held).
        registry.set(name, holding=["buffer_lock"], waiting_for="stats_lock")
        stats_lock.acquire()
        registry.set(name, holding=["buffer_lock", "stats_lock"], waiting_for=None)
        stats["produced"] += 1
        stats_lock.release()
        registry.set(name, holding=["buffer_lock"], waiting_for=None)
        buffer_lock.release()
        registry.set(name, holding=[], waiting_for=None)

        if cadence_s:
            time.sleep(cadence_s)
    registry.set(name, role="producer", holding=[], waiting_for="finished")


def consumer_body(name, buffer, buffer_lock, stats_lock, count, stats, registry,
                  service_s, stop):
    for _ in range(count):
        if stop.is_set():
            return
        # FLAWED ORDER: stats_lock first ...
        stats_lock.acquire()
        registry.set(name, role="consumer", holding=["stats_lock"], waiting_for=None)

        # ... then buffer_lock - the inverted order that closes the circular wait.
        registry.set(name, holding=["stats_lock"], waiting_for="buffer_lock")
        while not buffer_lock.acquire(timeout=0.25):
            if stop.is_set():
                registry.set(name, holding=["stats_lock"], waiting_for="abandoned")
                stats_lock.release()
                registry.set(name, holding=[], waiting_for=None)
                return
            registry.set(name, holding=["stats_lock"], waiting_for="buffer_lock")

        # ... and "wait for an item" is evaluated while holding BOTH locks.
        while not buffer:
            if stop.is_set():
                buffer_lock.release()
                stats_lock.release()
                registry.set(name, holding=[], waiting_for=None)
                return
            registry.set(name, holding=["stats_lock", "buffer_lock"],
                         waiting_for="buffer item (needs a producer to push)")
            time.sleep(0.001)

        buffer.pop(0)
        stats["consumed"] += 1
        buffer_lock.release()
        registry.set(name, holding=["stats_lock"], waiting_for=None)
        stats_lock.release()
        registry.set(name, holding=[], waiting_for=None)

        if service_s:
            time.sleep(service_s)
    registry.set(name, role="consumer", holding=[], waiting_for="finished")


def watchdog_body(stop_event, registry, stats, buffer, capacity, report, stall_s, max_s):
    """Flag stalled progress; no manual kill needed."""
    start = time.perf_counter()
    last_progress = None
    last_change = time.perf_counter()
    while not stop_event.wait(0.2):
        now = time.perf_counter()
        progress = stats["produced"] + stats["consumed"]
        if progress != last_progress:
            last_progress = progress
            last_change = now
            continue
        stalled = now - last_change
        if stalled >= stall_s:
            report["deadlock"] = True
            report["stalled_s"] = round(stalled, 3)
            report["progress"] = {"produced": stats["produced"],
                                  "consumed": stats["consumed"],
```

```
                                "buffer_occupancy": len(buffer)}
            report["snapshot"] = registry.snapshot()
            report["live_threads"] = sorted(
                t.name for t in threading.enumerate()
                if t is not threading.current_thread() and t.is_alive()
            )
            return
        if now - start >= max_s:
            report["timeout_without_deadlock"] = True
            report["progress"] = {"produced": stats["produced"],
                                  "consumed": stats["consumed"],
                                  "buffer_occupancy": len(buffer)}
            return



def run_once(capacity, producers, consumers, items_per_producer, stall_s, max_s):
    buffer = []
    buffer_lock = threading.Lock()
    stats_lock = threading.Lock()
    stats = {"produced": 0, "consumed": 0}
    registry = Registry()
    stop_event = threading.Event()
    report = {"deadlock": False, "capacity": capacity, "producers": producers,
              "consumers": consumers, "items_per_producer": items_per_producer}


    prod_threads = [
        threading.Thread(
            target=producer_body,
            args=(f"P{p}", buffer, buffer_lock, stats_lock, capacity,
                  list(range(p * items_per_producer, (p + 1) * items_per_producer)),
                  stats, registry, 0.0005, stop_event),
            name=f"P{p}", daemon=True,
        )
        for p in range(producers)
    ]
    total_items = producers * items_per_producer
    per_consumer = total_items // consumers
    cons_threads = [
        threading.Thread(
            target=consumer_body,
            args=(f"C{c}", buffer, buffer_lock, stats_lock, per_consumer, stats,
                  registry, 0.001, stop_event),
            name=f"C{c}", daemon=True,
        )
        for c in range(consumers)
    ]


    watch = threading.Thread(target=watchdog_body,
                             args=(stop_event, registry, stats, buffer, capacity, report,
                                   stall_s, max_s),
                             name="watchdog")
    start = time.perf_counter()
    for t in prod_threads + cons_threads:
        t.start()
    watch.start()
    watch.join()
    stop_event.set()


    if report["deadlock"]:
        report["elapsed_s"] = round(time.perf_counter() - start, 3)
        return report


    for t in prod_threads + cons_threads:
        t.join(timeout=10)
    report["completed"] = stats["consumed"] >= total_items
    report["elapsed_s"] = round(time.perf_counter() - start, 3)
    return report



def print_deadlock(report):
    print("=" * 78)
    print(f"WATCHDOG: DEADLOCK DETECTED - no progress for {report['stalled_s']:.2f}s "
          f"(run age {report['elapsed_s']:.2f}s)")
    prog = report["progress"]
    print(f"  items produced = {prog['produced']}, items consumed = {prog['consumed']}, "
          f"buffer occupancy = {prog['buffer_occupancy']} / capacity {report['capacity']}")
    print("  LOCK-WAIT SNAPSHOT (task -> holds -> waits for):")
    for entry in report["snapshot"]:
        holds = "+".join(entry.get("holding", [])) or "-"
        print(f"    {entry['name']:<3} role={entry.get('role', '?'):<9} "
              f"holds=[{holds:<26}] waiting_for={entry.get('waiting_for')}")
    print(f"  live task threads: {', '.join(report['live_threads'])}")
    print("  Coffman conditions in force: mutual exclusion (two exclusive locks),")
```

```
    print("  hold-and-wait (each role holds one lock while requesting the other),")
    print("  no preemption (a spinning holder cannot be divested of its lock),")
    print("  circular wait (P holds buffer_lock and needs stats_lock;")
    print("  C holds stats_lock and needs buffer_lock).")
    print("=" * 78)



def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--capacity", type=int, default=4)
    parser.add_argument("--producers", type=int, default=3)
    parser.add_argument("--consumers", type=int, default=3)
    parser.add_argument("--items", type=int, default=240,
                        help="total items to produce across all producers")
    parser.add_argument("--stall-seconds", type=float, default=2.0)
    parser.add_argument("--max-seconds", type=float, default=25.0)
    parser.add_argument("--attempts", type=int, default=5)
    args = parser.parse_args()

    high_resolution_timer()
    items_per_producer = max(1, args.items // args.producers)

    summary = {"version": "flawed", "capacity": args.capacity,
               "producers": args.producers, "consumers": args.consumers,
               "items_per_producer": items_per_producer, "attempts": []}
    deadlocked = False
    for attempt in range(1, args.attempts + 1):
        print(f"\n--- attempt {attempt}: capacity={args.capacity} "
              f"producers={args.producers} consumers={args.consumers} "
              f"items/producer={items_per_producer} ---")
        report = run_once(args.capacity, args.producers, args.consumers,
                          items_per_producer, args.stall_seconds, args.max_seconds)
        if report["deadlock"]:
            print_deadlock(report)
        elif report.get("timeout_without_deadlock"):
            print("  watchdog: progress continued for the whole window - no stall")
        else:
            print(f"  run finished without deadlock in {report['elapsed_s']}s "
                  f"(consumed={report['progress']['consumed']})")

        summary["attempts"].append({
            "attempt": attempt,
            "deadlock": report["deadlock"],
            "stalled_s": report.get("stalled_s"),
            "elapsed_s": report.get("elapsed_s"),
            "progress": report.get("progress"),
            "snapshot": [
                {"task": s["name"], "role": s.get("role"), "holding": s.get("holding"),
                 "waiting_for": s.get("waiting_for")}
                for s in report.get("snapshot", [])
            ],
            "live_threads": report.get("live_threads"),
        })
        if report["deadlock"]:
            deadlocked = True
            break

    stuck = summary["attempts"][-1] if deadlocked else None
    summary["reproduced"] = deadlocked
    summary["reproduced_on_attempt"] = len(summary["attempts"]) if deadlocked else None
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    out = RESULTS_DIR / "mp32_deadlock_evidence.json"
    out.write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print(f"\nreproduced={deadlocked} "
          f"attempt={summary['reproduced_on_attempt']}  evidence -> {out}")
    if stuck:
        print("stuck tasks: " + ", ".join(
            f"{s['task']}({s['role']}) holds {s['holding']} wants {s['waiting_for']}"
            for s in stuck["snapshot"] if s["holding"]))
    return 2 if deadlocked else 1



if __name__ == "__main__":
    sys.exit(main())
```

B.2 buffer_fixed.py - global lock ordering

File: mp32/buffer_fixed.py (229 lines)

```
"""Mini Project 3.2 (corrected version): lock-ordered bounded buffer.
```

```
The only structural change from buffer_flawed.py is the lock discipline:

  * EVERY task acquires buffer_lock before stats_lock, never the reverse, so circular
    wait (Coffman condition 4) cannot form.
  * The blocking conditions ("wait for space", "wait for an item") are evaluated on a
    threading.Condition bound to buffer_lock, so a waiting task RELEASES buffer_lock
    instead of spinning while holding it - it no longer holds-and-waits.

The watchdog is kept in place and must stay silent. Per-item producer and consumer
blocking times and overall throughput are recorded for the capacity experiment.

Usage:
    python buffer_fixed.py --capacity 5 --producers 3 --consumers 3 --items 240
"""

import argparse
import json
import statistics
import sys
import threading
import time
from contextlib import contextmanager
from pathlib import Path

RESULTS_DIR = Path(__file__).resolve().parent.parent / "results"



def high_resolution_timer():
    if sys.platform == "win32":
        try:
            __import__("ctypes").windll.winmm.timeBeginPeriod(1)
        except Exception:
            pass



@contextmanager
def both_locks(buffer_lock, stats_lock):
    """The agreed global order: buffer_lock first, then stats_lock, always."""
    buffer_lock.acquire()
    try:
        stats_lock.acquire()
        try:
            yield
        finally:
            stats_lock.release()
    finally:
        buffer_lock.release()



def producer_body(cond, stats_lock, buffer, capacity, item_ids, stats, waits,
                  burst, burst_pause_s):
    for index, item in enumerate(item_ids):
        t0 = time.perf_counter()
        with cond:                                  # holds buffer_lock
            while len(buffer) >= capacity:          # wait for space, lock released
                cond.wait()
            buffer.append(item)
            waited = time.perf_counter() - t0
            waits.append(waited)
            with stats_lock:                        # same order as every other task
                stats["produced"] += 1
                stats["producer_blocked"] += 1 if waited > 0.0005 else 0
            cond.notify_all()                       # wake consumers after publishing
        # Bursty arrival: emit `burst` items back to back, then go quiet for a while.
        # This is the traffic pattern in which buffer capacity actually buys something.
        if burst and (index + 1) % burst == 0:
            time.sleep(burst_pause_s)



def consumer_body(cond, stats_lock, buffer, count, stats, waits, service_s):
    for _ in range(count):
        t0 = time.perf_counter()
        with cond:
            while not buffer:                       # wait for an item, lock released
                cond.wait()
            buffer.pop(0)
            waited = time.perf_counter() - t0
            waits.append(waited)
            with stats_lock:
```

```
                stats["consumed"] += 1
                stats["consumer_blocked"] += 1 if waited > 0.0005 else 0
            cond.notify_all()                       # wake producers after freeing space
        if service_s:
            time.sleep(service_s)



def watchdog_body(stop_event, stats, buffer, report, stall_s, max_s, total_items):
    start = time.perf_counter()
    last = None
    last_change = time.perf_counter()
    while not stop_event.wait(0.2):
        now = time.perf_counter()
        progress = stats["produced"] + stats["consumed"]
        if progress != last:
            last = progress
            last_change = now
        elif now - last_change >= stall_s:
            report["deadlock"] = True
            report["stalled_s"] = round(now - last_change, 3)
            return
        if now - start >= max_s:
            report["timeout_without_deadlock"] = True
            return



def run_once(capacity=5, producers=3, consumers=3, total_items=240, service_s=0.001,
             cadence_s=0.0005, stall_s=3.0, max_s=60.0, burst=0, burst_pause_s=0.008):
    buffer = []
    buffer_lock = threading.Lock()
    stats_lock = threading.Lock()
    cond = threading.Condition(buffer_lock)
    stats = {"produced": 0, "consumed": 0, "producer_blocked": 0, "consumer_blocked": 0}
    producer_waits, consumer_waits = [], []
    report = {"deadlock": False}
    stop_event = threading.Event()

    per_producer = total_items // producers
    per_consumer = total_items // consumers

    prod_threads = [
        threading.Thread(target=producer_body,
                         args=(cond, stats_lock, buffer, capacity,
                               list(range(p * per_producer, (p + 1) * per_producer)),
                               stats, producer_waits, burst, burst_pause_s),
                         name=f"P{p}")
        for p in range(producers)
    ]
    cons_threads = [
        threading.Thread(target=consumer_body,
                         args=(cond, stats_lock, buffer, per_consumer, stats,
                               consumer_waits, service_s),
                         name=f"C{c}")
        for c in range(consumers)
    ]
    watch = threading.Thread(target=watchdog_body,
                             args=(stop_event, stats, buffer, report, stall_s, max_s,
                                   total_items),
                             name="watchdog")

    start = time.perf_counter()
    for t in prod_threads + cons_threads:
        t.start()
    watch.start()
    for t in prod_threads + cons_threads:
        t.join()
    elapsed = time.perf_counter() - start
    stop_event.set()
    watch.join()

    def summarise(values):
        if not values:
            return {"mean_ms": 0.0, "max_ms": 0.0, "count_blocked": 0}
        return {
            "mean_ms": round(statistics.fmean(values) * 1000, 3),
            "max_ms": round(max(values) * 1000, 3),
            "p95_ms": round(sorted(values)[min(len(values) - 1,
                                               int(0.95 * (len(values) - 1)))] * 1000, 3),
        }

    result = {
        "capacity": capacity,
        "producers": producers,
```

```
        "consumers": consumers,
        "total_items": producers * per_producer,
        "produced": stats["produced"],
        "consumed": stats["consumed"],
        "balanced": stats["produced"] == stats["consumed"],
        "elapsed_s": round(elapsed, 3),
        "throughput_items_per_s": round(stats["consumed"] / elapsed, 1),
        "producer_wait": summarise(producer_waits),
        "consumer_wait": summarise(consumer_waits),
        "producer_blocked": stats["producer_blocked"],
        "consumer_blocked": stats["consumer_blocked"],
        "deadlock": report["deadlock"],
        "timeout_without_deadlock": report.get("timeout_without_deadlock", False),
    }
    return result



def print_result(result):
    print(f"capacity={result['capacity']} producers={result['producers']} "
          f"consumers={result['consumers']} items={result['total_items']}")
    if result["deadlock"]:
        print("  WATCHDOG: DEADLOCK DETECTED (fix failed)")
    else:
        print("  watchdog stayed silent - no stall, fix verified")
    print(f"  produced={result['produced']} consumed={result['consumed']} "
          f"balanced={result['balanced']}")
    print(f"  elapsed={result['elapsed_s']}s  throughput={result['throughput_items_per_s']} items/s")
    pw, cw = result["producer_wait"], result["consumer_wait"]
    print(f"  producer wait for space: mean={pw['mean_ms']:.3f}ms "
          f"p95={pw.get('p95_ms', 0):.3f}ms max={pw['max_ms']:.3f}ms "
          f"(blocked {result['producer_blocked']} times)")
    print(f"  consumer wait for items: mean={cw['mean_ms']:.3f}ms "
          f"p95={cw.get('p95_ms', 0):.3f}ms max={cw['max_ms']:.3f}ms "
          f"(blocked {result['consumer_blocked']} times)")



def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--capacity", type=int, default=5)
    parser.add_argument("--producers", type=int, default=3)
    parser.add_argument("--consumers", type=int, default=3)
    parser.add_argument("--items", type=int, default=240)
    parser.add_argument("--service-ms", type=float, default=1.0)
    parser.add_argument("--burst", type=int, default=0,
                        help="items produced back-to-back before pausing (0 = steady flow)")
    parser.add_argument("--burst-pause-ms", type=float, default=8.0)
    parser.add_argument("--json", action="store_true", help="dump machine-readable result")
    args = parser.parse_args()

    high_resolution_timer()
    result = run_once(capacity=args.capacity, producers=args.producers,
                      consumers=args.consumers, total_items=args.items,
                      service_s=args.service_ms / 1000.0, burst=args.burst,
                      burst_pause_s=args.burst_pause_ms / 1000.0)
    print_result(result)
    if args.json:
        print(json.dumps(result, indent=2))
        RESULTS_DIR.mkdir(parents=True, exist_ok=True)
        (RESULTS_DIR / "mp32_fixed_run.json").write_text(
            json.dumps(result, indent=2), encoding="utf-8")
    return 0



if __name__ == "__main__":
    sys.exit(main())
```

B.3 capacity_sweep.py - throughput versus capacity

File: mp32/capacity_sweep.py (136 lines)

```
"""Mini Project 3.2, steps 6-7: throughput and blocking time versus buffer capacity.


Runs the corrected (lock-ordered) simulation over several buffer capacities with the
task counts and item totals held fixed, tabulates throughput and average blocking time,
identifies the point of diminishing returns, and plots the result.
"""


import csv
import json
import statistics
```

```
import sys
from pathlib import Path


HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))


import buffer_fixed  # noqa: E402
import matplotlib  # noqa: E402


matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402


RESULTS_DIR = HERE.parent / "results"
FIGURES_DIR = HERE.parent / "figures"


CAPACITIES = [1, 2, 5, 10, 25, 50]
PRODUCERS = 3
CONSUMERS = 3
TOTAL_ITEMS = 900
SERVICE_MS = 0.5
REPEATS = 5
BURST = 40                 # items produced back-to-back before the producer goes quiet
BURST_PAUSE_MS = 8.0       # arrival gap that lets a small buffer drain



def main():
    buffer_fixed.high_resolution_timer()
    rows = []
    for capacity in CAPACITIES:
        runs = [
            buffer_fixed.run_once(capacity=capacity, producers=PRODUCERS,
                                  consumers=CONSUMERS, total_items=TOTAL_ITEMS,
                                  service_s=SERVICE_MS / 1000.0, burst=BURST,
                                  burst_pause_s=BURST_PAUSE_MS / 1000.0)
            for _ in range(REPEATS)
        ]
        deadlocks = sum(r["deadlock"] for r in runs)
        row = {
            "capacity": capacity,
            "throughput_items_per_s": round(
                statistics.median(r["throughput_items_per_s"] for r in runs), 1),
            "elapsed_s": round(statistics.fmean(r["elapsed_s"] for r in runs), 3),
            "producer_wait_mean_ms": round(
                statistics.fmean(r["producer_wait"]["mean_ms"] for r in runs), 3),
            "consumer_wait_mean_ms": round(
                statistics.fmean(r["consumer_wait"]["mean_ms"] for r in runs), 3),
            "total_blocking_mean_ms": round(
                statistics.fmean(r["producer_wait"]["mean_ms"] +
                                 r["consumer_wait"]["mean_ms"] for r in runs), 3),
            "producer_blocked_events": int(
                statistics.fmean(r["producer_blocked"] for r in runs)),
            "consumer_blocked_events": int(
                statistics.fmean(r["consumer_blocked"] for r in runs)),
            "max_producer_wait_ms": round(
                max(r["producer_wait"]["max_ms"] for r in runs), 3),
            "max_consumer_wait_ms": round(
                max(r["consumer_wait"]["max_ms"] for r in runs), 3),
            "deadlocks": deadlocks,
            "balanced": all(r["balanced"] for r in runs),
        }
        rows.append(row)
        print(f"capacity={capacity:>3}  throughput={row['throughput_items_per_s']:>8.1f} "
              f"items/s  producer wait={row['producer_wait_mean_ms']:>7.3f} ms  "
              f"consumer wait={row['consumer_wait_mean_ms']:>7.3f} ms  "
              f"blocked(P/C)={row['producer_blocked_events']}/"
              f"{row['consumer_blocked_events']}  deadlocks={deadlocks}")


    # marginal throughput gain per added slot, to locate diminishing returns
    print("\nmarginal gain per extra buffer slot:")
    for prev, cur in zip(rows, rows[1:]):
        step = cur["capacity"] - prev["capacity"]
        gain = cur["throughput_items_per_s"] - prev["throughput_items_per_s"]
        print(f"  {prev['capacity']:>2} -> {cur['capacity']:>2}: "
              f"{gain:+8.1f} items/s over {step} slots "
              f"= {gain / step:+7.2f} items/s per slot")


    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    FIGURES_DIR.mkdir(parents=True, exist_ok=True)


    csv_path = RESULTS_DIR / "mp32_capacity_sweep.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0].keys()))
```

```
        writer.writeheader()
        writer.writerows(rows)

    (RESULTS_DIR / "mp32_capacity_sweep.json").write_text(
        json.dumps({"producers": PRODUCERS, "consumers": CONSUMERS,
                    "total_items": TOTAL_ITEMS, "service_ms": SERVICE_MS,
                    "repeats": REPEATS, "burst": BURST,
                    "burst_pause_ms": BURST_PAUSE_MS, "rows": rows}, indent=2),
        encoding="utf-8")

    caps = [r["capacity"] for r in rows]
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8.2, 6.2), sharex=True)
    ax1.plot(caps, [r["throughput_items_per_s"] for r in rows], "o-", color="#1f4e79",
             lw=2, label="throughput")
    ax1.set_ylabel("throughput (items/s)")
    ax1.set_xscale("log")
    ax1.grid(alpha=0.3)
    ax1.legend(loc="lower right")
    ax1.set_title("MP3.2 corrected producer-consumer: throughput and blocking\n"
                  f"{PRODUCERS} producers / {CONSUMERS} consumers, "
                  f"{TOTAL_ITEMS} items, {SERVICE_MS:.1f} ms service time")

    ax2.plot(caps, [r["producer_wait_mean_ms"] for r in rows], "s-", color="#c0504d",
             lw=2, label="producer wait for space")
    ax2.plot(caps, [r["consumer_wait_mean_ms"] for r in rows], "^-", color="#4f8a3b",
             lw=2, label="consumer wait for items")
    ax2.set_ylabel("mean blocking time (ms)")
    ax2.set_xlabel("buffer capacity (items)")
    ax2.set_xscale("log")
    ax2.set_xticks(caps)
    ax2.set_xticklabels(caps)
    ax2.grid(alpha=0.3)
    ax2.legend(loc="best")
    fig.tight_layout()
    png = FIGURES_DIR / "fig323_throughput_vs_capacity.png"
    fig.savefig(png, dpi=170)
    plt.close(fig)

    print(f"\nwrote {csv_path}")
    print(f"wrote {png}")



if __name__ == "__main__":
    main()
```

Appendix C - Mini Project 3.3 source

C.1 interrupt_latency_sim.py - priority interrupt simulator

File: mp33/interrupt_latency_sim.py (438 lines)

```
"""Mini Project 3.3: priority-based interrupt latency simulator.


Models a single simulated CPU, several periodic interrupt sources of different
priority, and the top-half / bottom-half split of Section 3.5:


  * each source thread waits for its period, stamps the assertion time, then takes the
    CPU lock and runs its top half. Top halves always get the CPU first, which is where
    interrupt latency (assertion -> first top-half instruction) is measured;
  * every top half enqueues a bottom-half job tagged with priority and assertion time
    into a heapq ready queue;
  * one worker thread executes the highest-priority ready bottom half, in slices. It
    releases the CPU between slices, so a newly arriving top half can take it and a
    higher-priority bottom half can displace the running one - the remainder of the
    displaced job is requeued (preemption, with its time counted).


Two distinct latencies are logged per occurrence, matching Section 3.5:
  interrupt latency = assertion -> top half starts on the CPU
  response latency  = assertion -> bottom half finishes


Usage:
    python interrupt_latency_sim.py --config baseline --seconds 20
    python interrupt_latency_sim.py --configs baseline,long_top,burst
    python interrupt_latency_sim.py --sweep 15,25,40,60,100,150 --seconds 10
"""


import argparse
import csv
import heapq
import json
import statistics
import sys
import threading
import time
from pathlib import Path


HERE = Path(__file__).resolve().parent
RESULTS_DIR = HERE.parent / "results"
FIGURES_DIR = HERE.parent / "figures"


SLICE_MS = 1.0                    # bottom-half scheduling quantum (context-switch analogue)


# name, period_ms, priority (1 = highest), top_half_ms, bottom_half_ms
BASELINE = [
    {"name": "IRQ_HIGH", "period_ms": 40.0, "priority": 1, "top_ms": 1.0, "bottom_ms": 6.0},
    {"name": "IRQ_MED", "period_ms": 100.0, "priority": 2, "top_ms": 1.0, "bottom_ms": 10.0},
    {"name": "IRQ_LOW", "period_ms": 250.0, "priority": 3, "top_ms": 1.0, "bottom_ms": 18.0},
]


CONFIGS = {
    # well-behaved: every top half is short (Section 3.5 guidance)
    "baseline": [dict(s) for s in BASELINE],
    # deliberate violation: the highest-priority source runs a 12 ms top half
    "long_top": [dict(BASELINE[0], top_ms=12.0), dict(BASELINE[1]), dict(BASELINE[2])],
    # arrival-rate stress: the highest-priority source fires 2.7x more often
    "burst": [dict(BASELINE[0], period_ms=15.0), dict(BASELINE[1]), dict(BASELINE[2])],
}



def high_resolution_timer(enable=True):
    if sys.platform != "win32":
        return False
    try:
        winmm = __import__("ctypes").windll.winmm
        if enable:
            winmm.timeBeginPeriod(1)
        else:
            winmm.timeEndPeriod(1)
        return True
    except Exception:
        return True
```

```
class Simulation:
    def __init__(self, sources, seconds, slice_ms=SLICE_MS, timeline_window_ms=None,
                 timeline_start_s=None):
        self.sources = sources
        self.seconds = seconds
        self.slice_s = slice_ms / 1000.0
        self.cpu_lock = threading.Lock()          # the single simulated CPU
        self.heap_lock = threading.Lock()
        self.ready = []                           # (priority, seq, job)
        self.seq = 0
        self.worker_wake = threading.Condition(threading.Lock())
        self.log = []
        self.log_lock = threading.Lock()
        self.timeline = []
        self.timeline_lock = threading.Lock()
        self.timeline_window_s = (timeline_window_ms or 0) / 1000.0
        self.timeline_start_s = timeline_start_s
        self.start_time = None
        self.stop = threading.Event()

    # ------------------------------------------------------------------ helpers
    def record(self, row):
        with self.log_lock:
            self.log.append(row)

    def record_timeline(self, kind, source, t0, t1, extra=""):
        if self.timeline_window_s <= 0:
            return
        rel0 = t0 - self.start_time
        rel1 = t1 - self.start_time
        if self.timeline_start_s is not None and rel1 < self.timeline_start_s:
            return
        if rel0 > self.timeline_start_s + self.timeline_window_s:
            return
        with self.timeline_lock:
            self.timeline.append({"kind": kind, "source": source,
                                  "start_ms": round((max(0.0, rel0 - self.timeline_start_s)) * 1000, 3),
                                  "end_ms": round((min(self.timeline_start_s + self.timeline_window_s, rel1)
                                                   - self.timeline_start_s) * 1000, 3),
                                  "detail": extra})

    def enqueue(self, job):
        with self.heap_lock:
            self.seq += 1
            heapq.heappush(self.ready, (job["priority"], self.seq, job))
        with self.worker_wake:
            self.worker_wake.notify()

    def pop_ready(self):
        with self.heap_lock:
            if self.ready:
                return heapq.heappop(self.ready)[2]
        return None

    def peek_priority(self):
        with self.heap_lock:
            return self.ready[0][0] if self.ready else None

    # ------------------------------------------------------------------ top half
    def source_thread(self, source):
        period_s = source["period_ms"] / 1000.0
        top_s = source["top_ms"] / 1000.0
        next_fire = self.start_time + period_s
        while not self.stop.is_set():
            delay = next_fire - time.perf_counter()
            if delay > 0:
                time.sleep(delay)
            next_fire += period_s
            assert_time = time.perf_counter()

            # interrupt latency: waiting to actually get the CPU / run the first
            # top-half instruction
            with self.cpu_lock:
                top_start = time.perf_counter()
                time.sleep(top_s)
                top_end = time.perf_counter()
            self.record_timeline("top", source["name"], top_start, top_end,
                                 f"{source['top_ms']:.1f} ms")
            self.enqueue({"priority": source["priority"], "source": source["name"],
                          "assert_time": assert_time, "top_start": top_start,
                          "top_end": top_end,
                          "remaining": source["bottom_ms"] / 1000.0,
                          "preemptions": 0, "cpu_waits": 0})
```

```
    # ------------------------------------------------------------ bottom half
    def worker_thread(self):
        while True:
            job = self.pop_ready()
            if job is None:
                if self.stop.is_set():
                    return                      # queue drained, all occurrences logged
                with self.worker_wake:
                    self.worker_wake.wait(0.002)
                continue

            while job["remaining"] > 0.0:
                # strict priority among bottom halves: yield to higher-priority work
                higher = self.peek_priority()
                if higher is not None and higher < job["priority"]:
                    job["preemptions"] += 1
                    self.enqueue(job)
                    break

                wait0 = time.perf_counter()
                acquired = self.cpu_lock.acquire(timeout=0.05)
                cpu_wait = time.perf_counter() - wait0
                if not acquired:
                    job["preemptions"] += 1
                    self.enqueue(job)
                    break
                job["cpu_waits"] += 1 if cpu_wait > 0.0008 else 0
                slice_s = min(self.slice_s, job["remaining"])
                t0 = time.perf_counter()
                time.sleep(slice_s)
                actual = time.perf_counter() - t0
                self.cpu_lock.release()
                self.record_timeline("bottom", job["source"], t0, t0 + actual,
                                     f"slice {actual * 1000:.2f} ms")
                job["remaining"] -= actual

            if job["remaining"] <= 0.0:
                finish = time.perf_counter()
                self.record({
                    "source": job["source"],
                    "priority": job["priority"],
                    "assert_rel_ms": round((job["assert_time"] - self.start_time) * 1000, 3),
                    "interrupt_latency_ms": round((job["top_start"] - job["assert_time"]) * 1000, 3),
                    "top_half_ms": round((job["top_end"] - job["top_start"]) * 1000, 3),
                    "response_latency_ms": round((finish - job["assert_time"]) * 1000, 3),
                    "preemptions": job["preemptions"],
                    "cpu_waits": job["cpu_waits"],
                })

    # ------------------------------------------------------------------ drive
    def run(self):
        self.start_time = time.perf_counter() + 0.05
        threads = [threading.Thread(target=self.source_thread, args=(s,), name=s["name"])
                   for s in self.sources]
        worker = threading.Thread(target=self.worker_thread, name="cpu-worker")
        for t in threads:
            t.start()
        worker.start()
        time.sleep(self.seconds)
        self.stop.set()
        for t in threads:
            t.join()
        with self.worker_wake:
            self.worker_wake.notify_all()
        worker.join(timeout=10.0)              # drains the ready queue before exiting


def p95(values):
    ordered = sorted(values)
    return ordered[min(len(ordered) - 1, int(round(0.95 * (len(ordered) - 1))))]


def summarise(rows):
    per_source = {}
    for row in rows:
        per_source.setdefault(row["source"], []).append(row)
    summary = {}
    for source, entries in per_source.items():
        response = [e["response_latency_ms"] for e in entries]
        interrupt = [e["interrupt_latency_ms"] for e in entries]
        summary[source] = {
            "occurrences": len(entries),
```

```
            "priority": entries[0]["priority"],
            "interrupt_latency_min_ms": round(min(interrupt), 3),
            "interrupt_latency_mean_ms": round(statistics.fmean(interrupt), 3),
            "interrupt_latency_max_ms": round(max(interrupt), 3),
            "response_min_ms": round(min(response), 3),
            "response_mean_ms": round(statistics.fmean(response), 3),
            "response_p95_ms": round(p95(response), 3),
            "response_max_ms": round(max(response), 3),
            "preempted_jobs": sum(1 for e in entries if e["preemptions"] > 0),
            "total_preemptions": sum(e["preemptions"] for e in entries),
        }
    return summary



def print_summary(config_name, summary, seconds):
    print(f"\n=== configuration '{config_name}' ({seconds:.0f}s run) ===")
    header = (f"{'source':<10}{'pri':>4}{'occ':>6}{'int min':>9}{'int mean':>10}"
              f"{'int max':>9}{'resp min':>10}{'resp mean':>11}{'resp p95':>10}"
              f"{'resp max':>10}{'preempt':>9}")
    print(header)
    for source in sorted(summary, key=lambda s: summary[s]["priority"]):
        st = summary[source]
        print(f"{source:<10}{st['priority']:>4}{st['occurrences']:>6}"
              f"{st['interrupt_latency_min_ms']:>9.2f}"
              f"{st['interrupt_latency_mean_ms']:>10.2f}"
              f"{st['interrupt_latency_max_ms']:>9.2f}"
              f"{st['response_min_ms']:>10.2f}{st['response_mean_ms']:>11.2f}"
              f"{st['response_p95_ms']:>10.2f}{st['response_max_ms']:>10.2f}"
              f"{st['total_preemptions']:>9}")



def write_outputs(payload):
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
    out = RESULTS_DIR / "mp33_results.json"
    out.write_text(json.dumps(payload, indent=2), encoding="utf-8")


    fields = ["source", "priority", "assert_rel_ms", "interrupt_latency_ms",
              "top_half_ms", "response_latency_ms", "preemptions", "cpu_waits"]
    log_path = RESULTS_DIR / "mp33_occurrence_log.csv"
    rows = list(payload.get("occurrence_log", []))
    for block in payload.get("configs", {}).values():
        rows.extend(block.get("occurrence_log", []))
    payload.pop("occurrence_log", None)
    if rows:
        with log_path.open("w", newline="", encoding="utf-8") as fh:
            writer = csv.DictWriter(fh, fieldnames=fields, extrasaction="ignore")
            writer.writeheader()
            writer.writerows(rows)
    print(f"\nwrote {out}\nwrote {log_path} ({len(rows)} occurrences)")



def run_configs(names, seconds, slice_ms, timeline_names=()):
    payload = {}
    for name in names:
        sim = Simulation([dict(s) for s in CONFIGS[name]], seconds, slice_ms=slice_ms,
                         timeline_window_ms=600.0 if name in timeline_names else 0.0,
                         timeline_start_s=2.0 if name in timeline_names else None)
        sim.run()
        summary = summarise(sim.log)
        print_summary(name, summary, seconds)
        payload[name] = {
            "sources": CONFIGS[name],
            "summary": summary,
            "occurrence_log": sim.log,
            "timeline": sim.timeline,
        }
    return payload



def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--config", type=str, default="baseline",
                        choices=sorted(CONFIGS))
    parser.add_argument("--configs", type=str, default=None,
                        help="comma separated list; runs each and compares")
    parser.add_argument("--sweep", type=str, default=None,
                        help="comma separated periods for the highest-priority source")
    parser.add_argument("--full", action="store_true",
                        help="run all three configurations and the arrival-rate sweep")
    parser.add_argument("--seconds", type=float, default=20.0)
    parser.add_argument("--sweep-seconds", type=float, default=10.0)
    parser.add_argument("--slice-ms", type=float, default=SLICE_MS)
    parser.add_argument("--timeline", action="store_true",
```

```
                        help="capture a representative window for the timeline plot")
    args = parser.parse_args()

    high_resolution_timer(True)
    payload = {"slice_ms": args.slice_ms, "configs": {}, "sweep": None}
    try:
        timeline_names = ()
        if args.full:
            names = sorted(CONFIGS)
            if args.timeline:
                timeline_names = ("baseline", "long_top")
        elif args.sweep:
            names = []
            payload["sweep"] = run_period_sweep(
                [float(p) for p in args.sweep.split(",")], args.sweep_seconds,
                args.slice_ms)
            print_period_sweep(payload["sweep"])
        else:
            names = ([args.config] if not args.configs
                     else [c.strip() for c in args.configs.split(",")])


        if names:
            payload["configs"] = run_configs(names, args.seconds, args.slice_ms,
                                             timeline_names)
        if args.full:
            payload["sweep"] = run_period_sweep(
                [15.0, 25.0, 40.0, 60.0, 100.0, 150.0], args.sweep_seconds,
                args.slice_ms)
            print_period_sweep(payload["sweep"])


        write_outputs(payload)
        timeline_payload = {n: b for n, b in payload["configs"].items() if b["timeline"]}
        if timeline_payload:
            write_timeline_figure(timeline_payload)
    finally:
        high_resolution_timer(False)



def run_period_sweep(periods, seconds, slice_ms):
    """Sweep the highest-priority source's arrival rate; watch the lowest one suffer."""
    rows = []
    for period in periods:
        sources = [dict(CONFIGS["baseline"][0], period_ms=period),
                   dict(CONFIGS["baseline"][1]), dict(CONFIGS["baseline"][2])]
        sim = Simulation(sources, seconds, slice_ms=slice_ms)
        sim.run()
        summary = summarise(sim.log)
        low = summary.get("IRQ_LOW", {})
        high = summary.get("IRQ_HIGH", {})
        med = summary.get("IRQ_MED", {})
        rows.append({
            "high_period_ms": period,
            "high_occurrences": high.get("occurrences", 0),
            "high_response_max_ms": high.get("response_max_ms"),
            "med_response_p95_ms": med.get("response_p95_ms"),
            "low_response_min_ms": low.get("response_min_ms"),
            "low_response_mean_ms": low.get("response_mean_ms"),
            "low_response_p95_ms": low.get("response_p95_ms"),
            "low_response_max_ms": low.get("response_max_ms"),
            "low_interrupt_max_ms": low.get("interrupt_latency_max_ms"),
            "low_preemptions": low.get("total_preemptions", 0),
            "low_occurrences": low.get("occurrences", 0),
        })
    return rows



def print_period_sweep(rows):
    print("\n=== arrival-rate sweep of IRQ_HIGH (lowest-priority victim: IRQ_LOW) ===")
    print(f"{'high period ms':>15}{'high occ':>10}{'low mean ms':>13}"
          f"{'low p95 ms':>12}{'low max ms':>12}{'low preempt':>12}")
    for r in rows:
        print(f"{r['high_period_ms']:>15.1f}{r['high_occurrences']:>10}"
              f"{r['low_response_mean_ms']:>13.2f}{r['low_response_p95_ms']:>12.2f}"
              f"{r['low_response_max_ms']:>12.2f}{r['low_preemptions']:>12}")



def write_timeline_figure(config_payload):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    FIGURES_DIR.mkdir(parents=True, exist_ok=True)
```

```
    fig, axes = plt.subplots(len(config_payload), 1, figsize=(9.4, 2.6 * len(config_payload)),
                             sharex=True)
    if len(config_payload) == 1:
        axes = [axes]
    order = {"IRQ_HIGH": 2, "IRQ_MED": 1, "IRQ_LOW": 0, "top": 3}
    colors = {"IRQ_HIGH": "#c0504d", "IRQ_MED": "#4f8a3b", "IRQ_LOW": "#1f4e79"}
    for ax, (name, block) in zip(axes, config_payload.items()):
        events = block.get("timeline", [])
        if not events:
            ax.set_ylabel(name)
            ax.text(0.02, 0.5, "no timeline captured", transform=ax.transAxes)
            continue
        for event in events:
            lane = order.get(event["source"], 1) if event["kind"] == "bottom" else 3
            ax.barh(lane, event["end_ms"] - event["start_ms"], left=event["start_ms"],
                    height=0.6 if lane < 3 else 0.35,
                    color=colors.get(event["source"], "#7f7f7f"),
                    alpha=0.85 if event["kind"] == "bottom" else 1.0,
                    edgecolor="black", linewidth=0.3)
        ax.set_yticks([0, 1, 2, 3])
        ax.set_yticklabels(["IRQ_LOW", "IRQ_MED", "IRQ_HIGH", "top half"], fontsize=8)
        ax.set_ylabel(name)
        ax.grid(axis="x", alpha=0.3)
        ax.set_title(f"'{name}' - representative 600 ms window "
                     "(sliced bars = preemption points)", fontsize=10)
    axes[-1].set_xlabel("time in window (ms)")
    fig.tight_layout()
    png = FIGURES_DIR / "fig333_timeline.png"
    fig.savefig(png, dpi=170)
    plt.close(fig)
    print(f"wrote {png}")



if __name__ == "__main__":
    main()
```